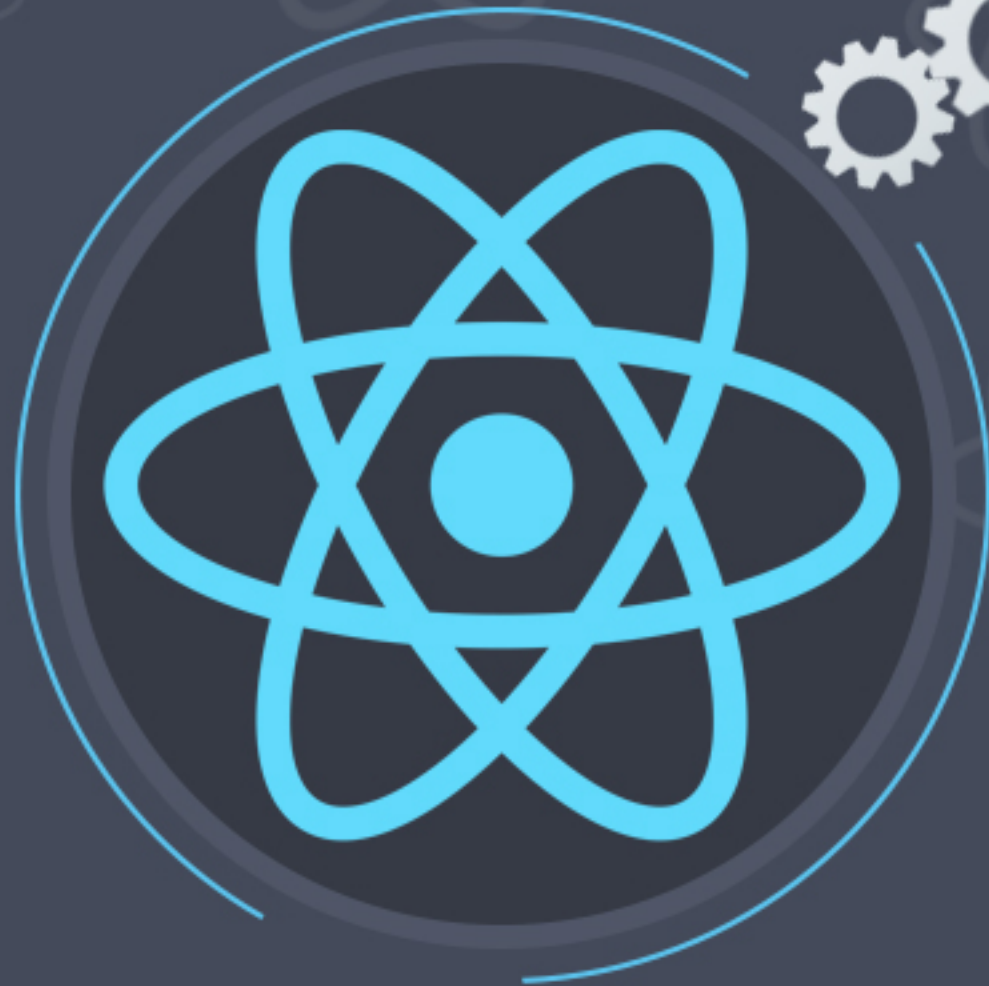




React.js



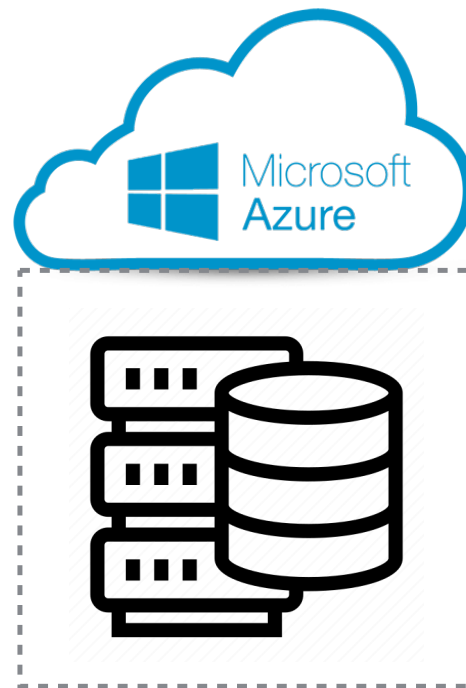
클라우드 기반 생성형 AI활용 웹개발 실무 프로젝트

클라우드 기반 생성형 AI 활용 웹개발자 과정


Azure AI services


OpenAI

Backend



 mongoDB®

 PostgreSQL

{RESTful API}

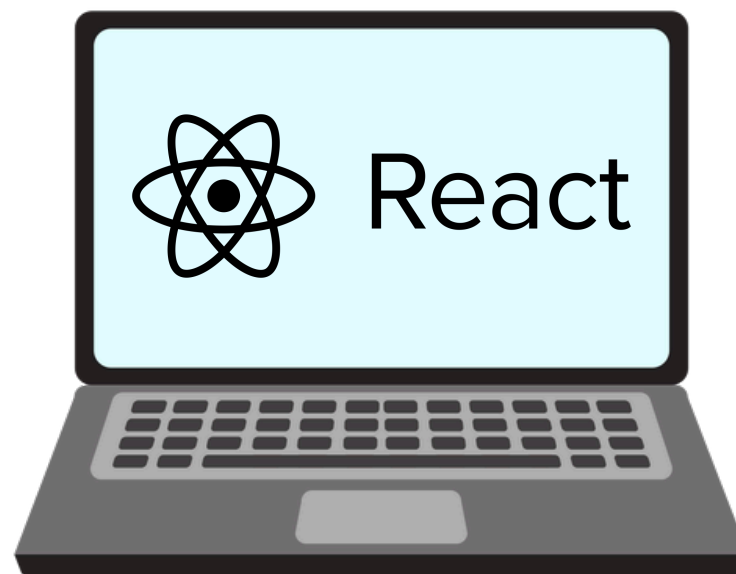
 node
JS

 git

 GitHub

 POSTMAN

Frontend



HTML


CSS

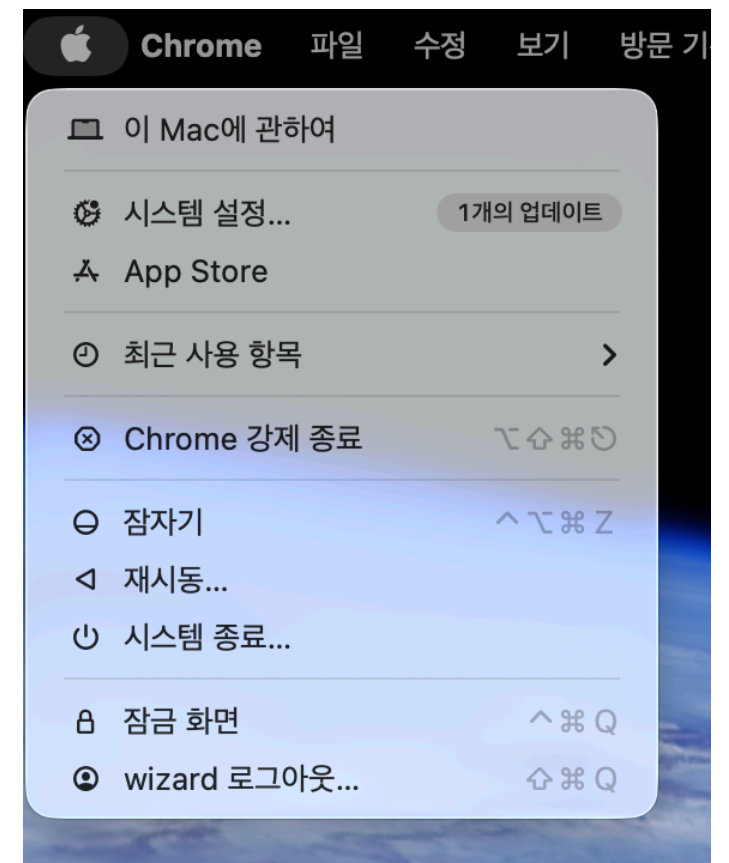

JS


NEXT.js

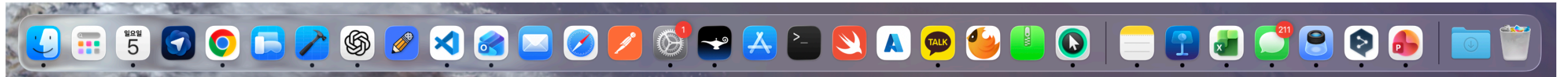
 AI

Mac 사용법

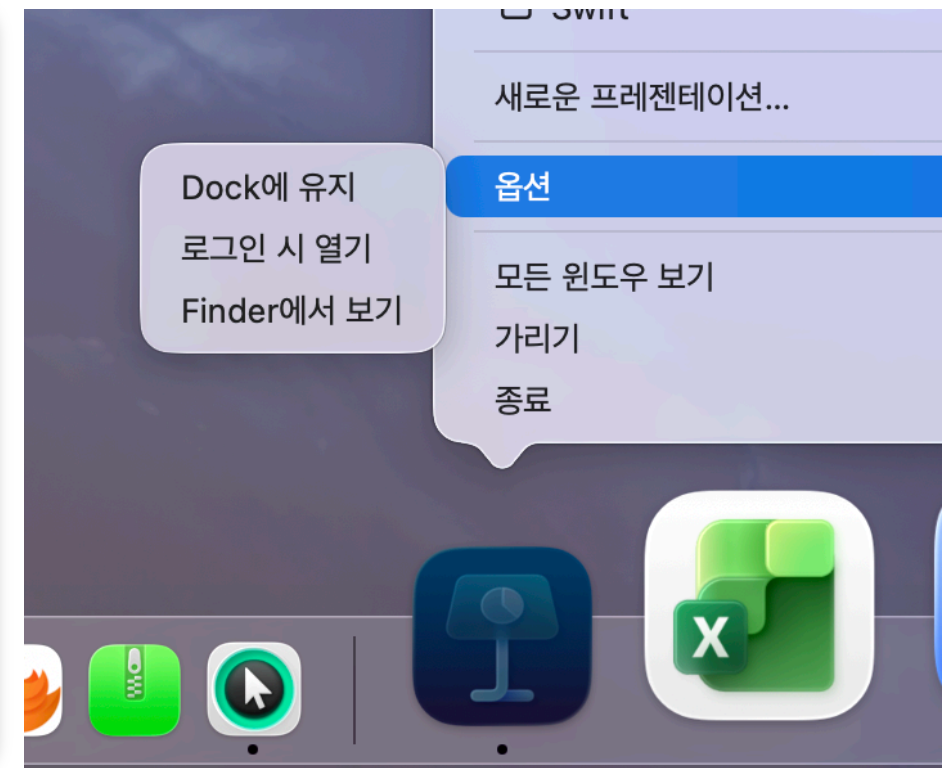
Mac 켜고 끄기



앱 실행하기

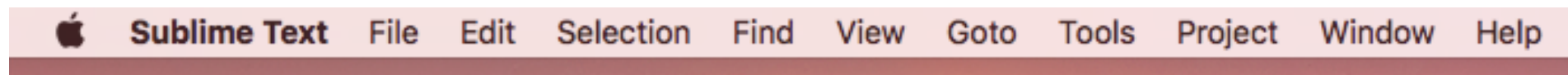
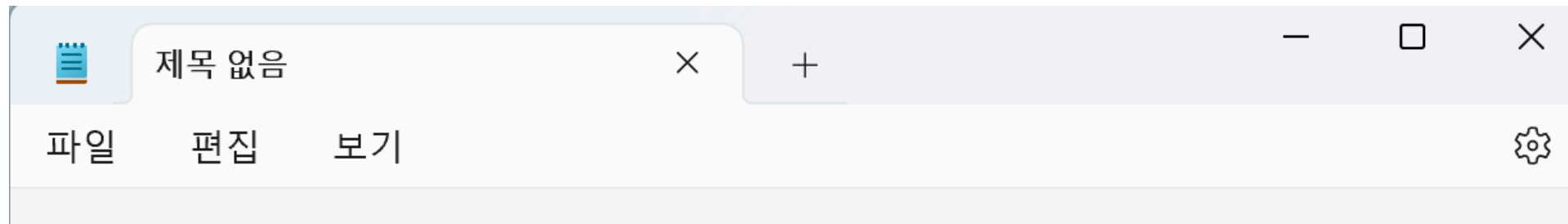


Dock에 있는 아이콘을 클릭해서 실행



Dock에 없는 프로그램은 앱 실행한후 아이콘 클릭
Dock에 고정하고 싶으면 옵션에서 Dock에 유지 혹은 드래그해서 이동

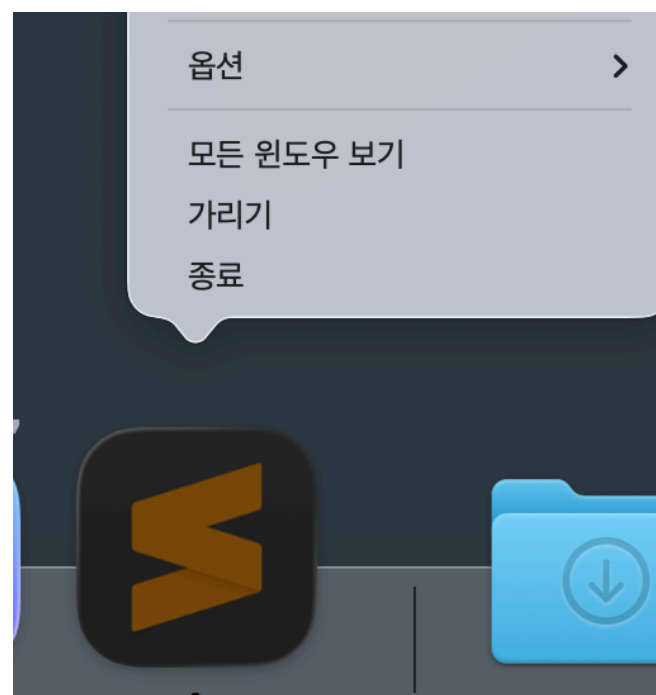
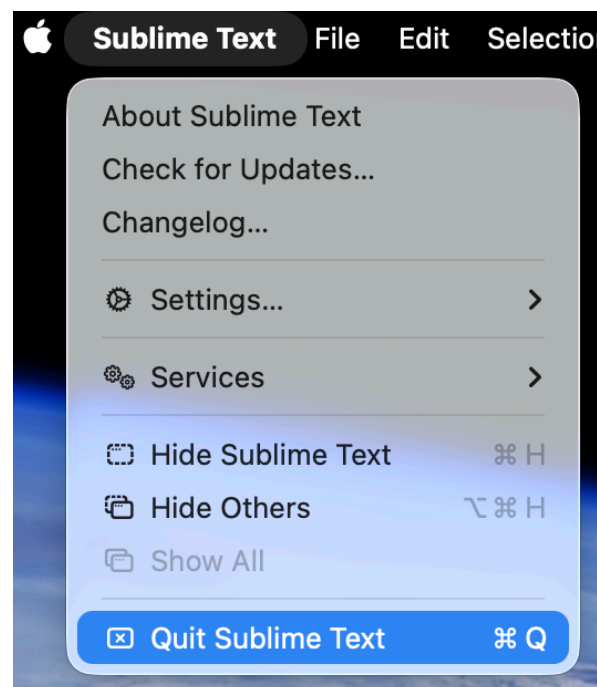
앱 메뉴 및 종료하기



윈도우와 달리 MacOS는 메뉴가 화면 상단에 고정 되어있음
활성화되어 있는 앱에 따라 메뉴가 변경

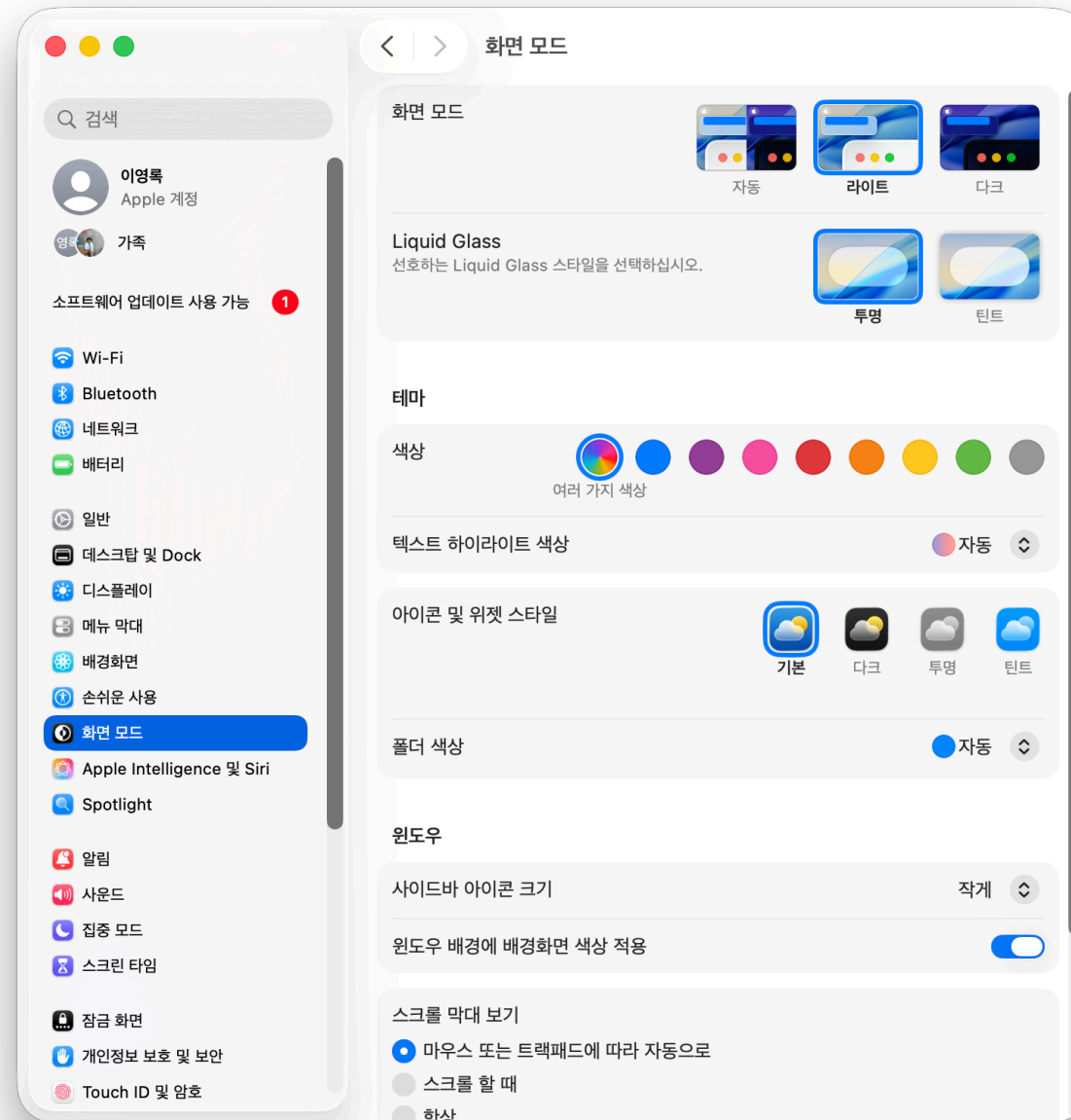


닫기-최소-최대 메뉴가 윈도우 왼쪽상단에 있음



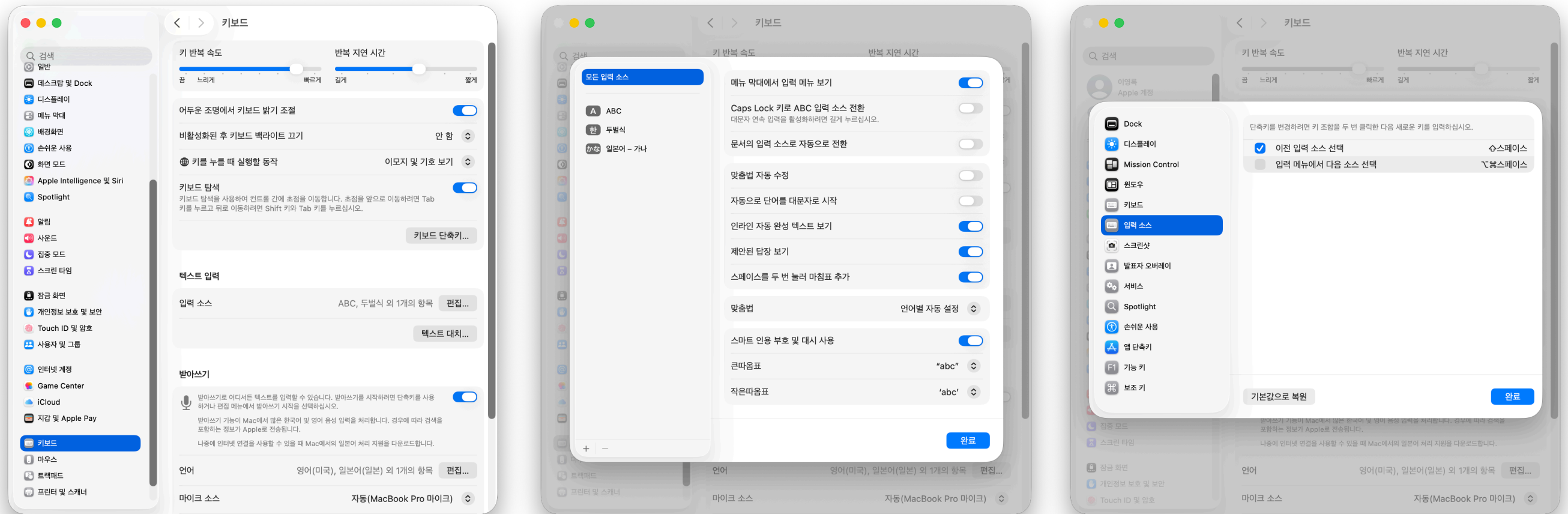
실행 종료를 할때는 앱 메뉴의
xxx 종료(cmd+Q)를 선택하거나
Dock에 있는 아이콘 위에서
오른쪽 버튼 누른후 종료 클릭

시스템 설정 메뉴

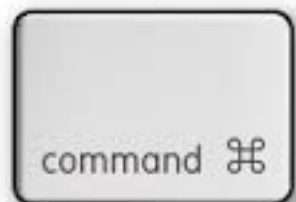


Dock에서 시스템 설정 아이콘 을 클릭 하거나
Apple 메뉴  > 시스템 설정을 선택

언어선택



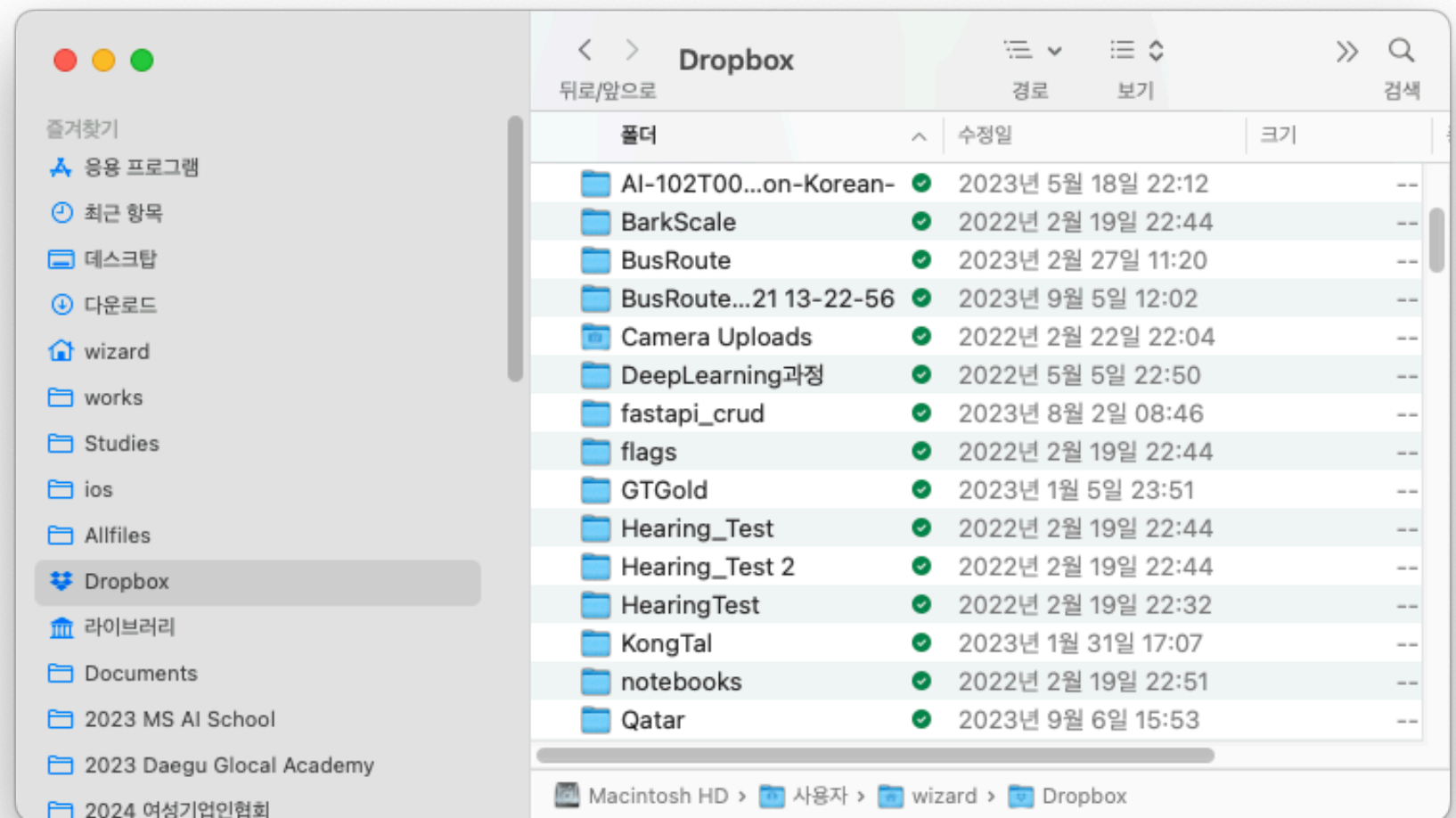
CapsLock 짧게, ctrl + space , shift + space



Windows의 control 키와 비슷한 역할

Finder

Windows 의 파일탐색기와 비슷한 역할



프로그래밍 언어

프로그래밍 언어

- 프로그래밍 언어는 컴퓨터 시스템을 구동시키는 소프트웨어를 작성하기 위한 언어
- 기계가 알 수 있는 건 이진수 뿐임
- 초창기에는 이진수로 프로그래밍함
- 사람이 알 수 있는 언어로 작성 후 기계가 이해할 수 있는 이진수로 바꿔주는 컴파일이라는 과정을 거침
- 고급(High level) 언어일수록 인간의 언어와 유사함



각종 프로그래밍 언어



Java™
1999



Visual Basic

Visual Basic



Kotlin



THE
C

PROGRAMMING
LANGUAGE



ALGOL



RUBY

Programming

Language

Language

Language

Language

Language

Language

Language

Language

Language

JavaScript



golang



Objective-C



Perl

Fortran⁷⁷



Haskell



프로그래밍 언어 분류

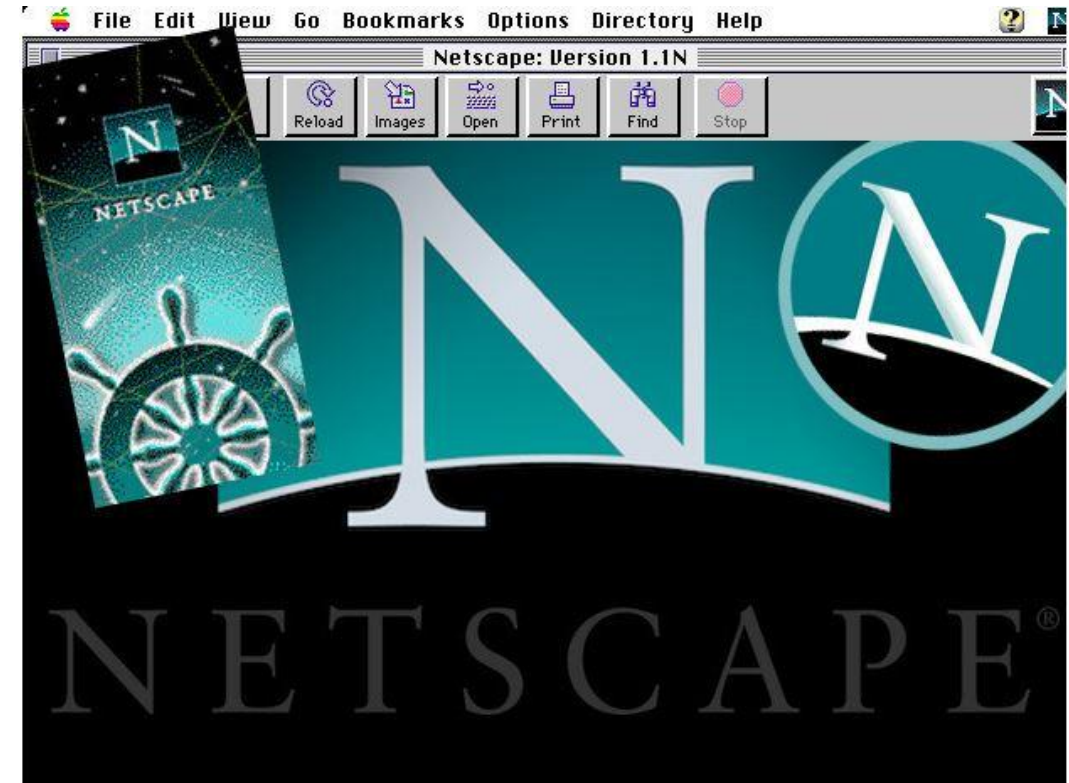
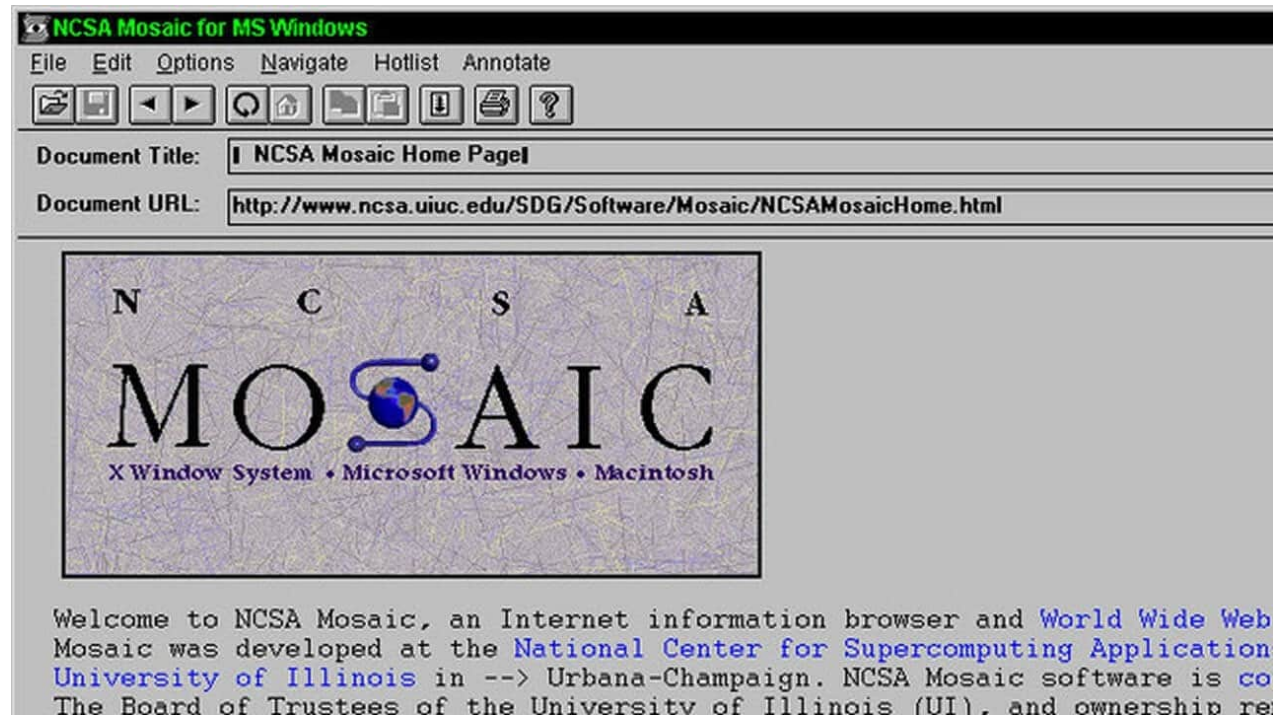
- 구조적 프로그래밍 : Algol, Fortran Basic, C
- 객체지향프로그래밍 : SmallTalk, C++, Java, Objective-c
- 함수형 프로그래밍 : Haskell, Lisp, Erlang
- 객체지향, 함수형 : Java, javascript, swift, python 등

명령어 해석 방식

	인터프리터 방식 interpreter	컴파일 방식 compile
정의	명령어들을 한 번에 한 줄씩 읽어들이어서 실행하는 방식	명령어를 기계어로 번역하는 방식
장점	컴파일 단계를 거칠 필요가 없다	일반적인 경우 속도가 더 빠르다
단점	실행 시간이 느리다	원시 프로그램의 크기가 크다면 상당한 시간이 소요된다
사용 언어	Python, Basic, Javascript, ...	C, PASCAL, swift ...
특징	약타입, 동적타입	강타입, 정적타입

JavaScript 소개

웹 브라우저의 등장

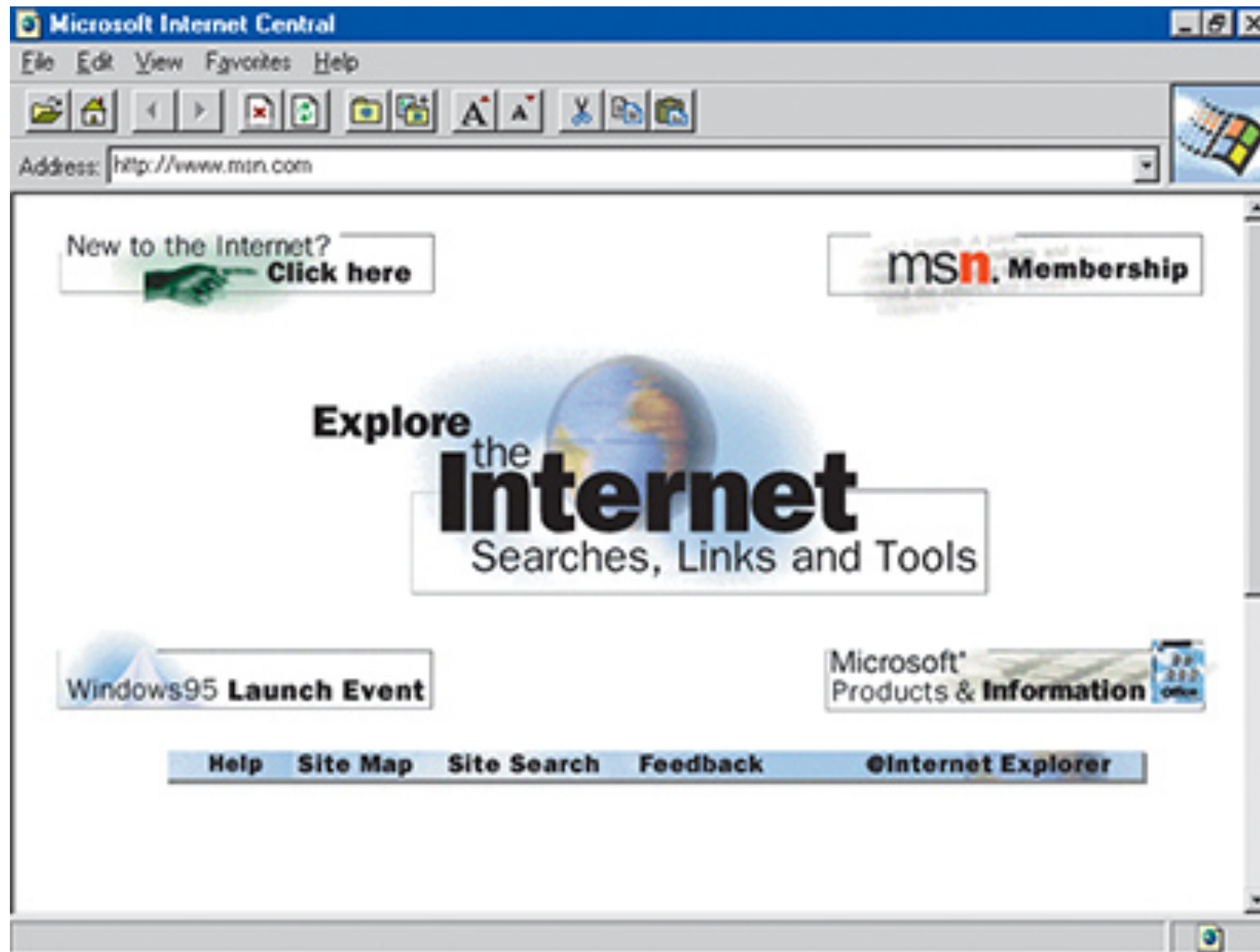


1993년 개인용 컴퓨터에서 사용할 수 있는 Mosaic Web Browser가 출시

1994년 마크 앤드리슨이 Netscape 출시, 80% 점유

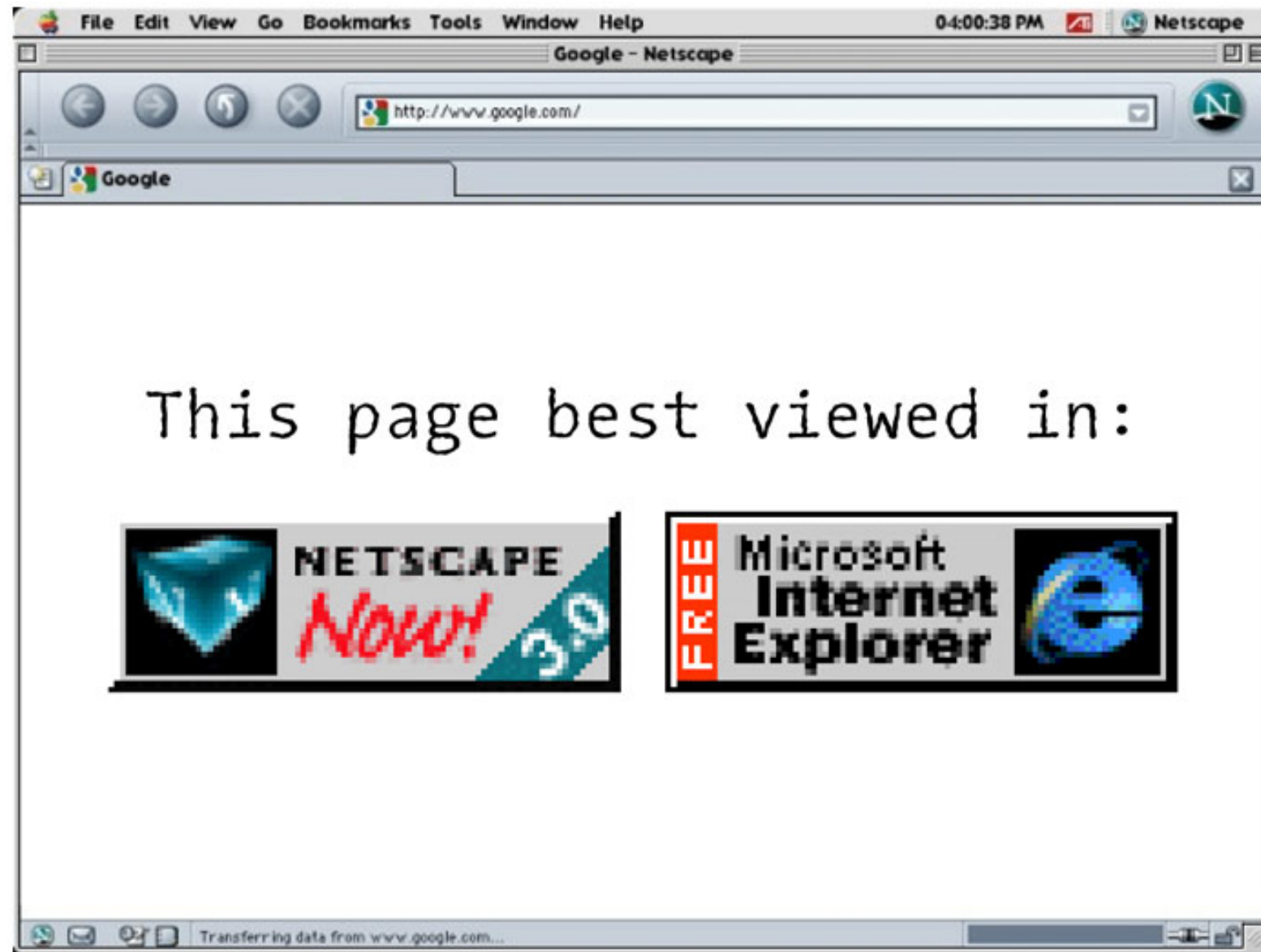
1995년 브랜든 아이크가 Live Script 개발=>JavaScript로 개명

Internet Explorer의 등장



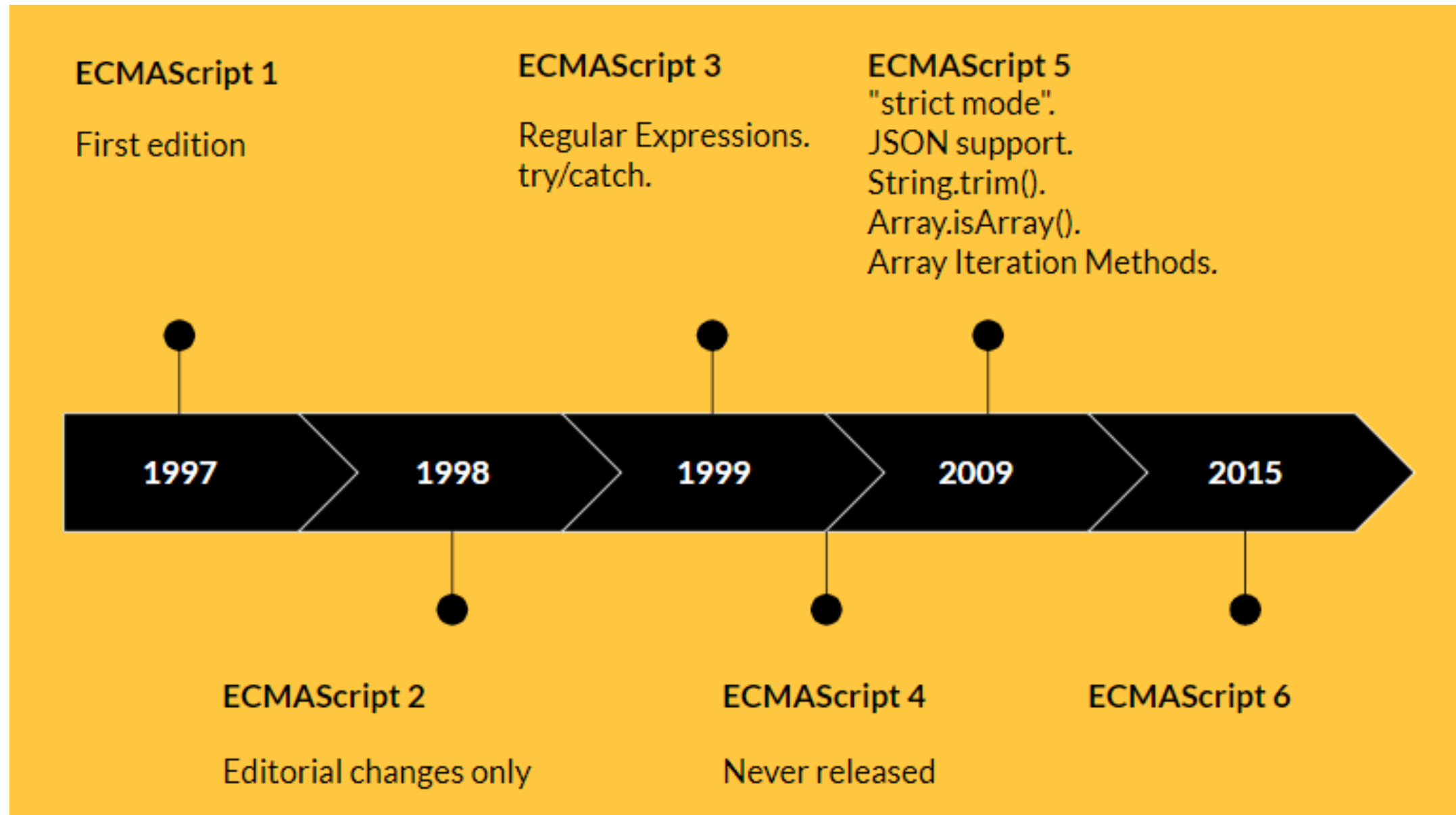
1995년 Netscape Navigator를 참조해 ie를 출시, JScript 내장

브라우저 전쟁



표준의 부재와 경쟁때문에 브라우저 별로 별도의 코드 작성 필요

ECMAScript



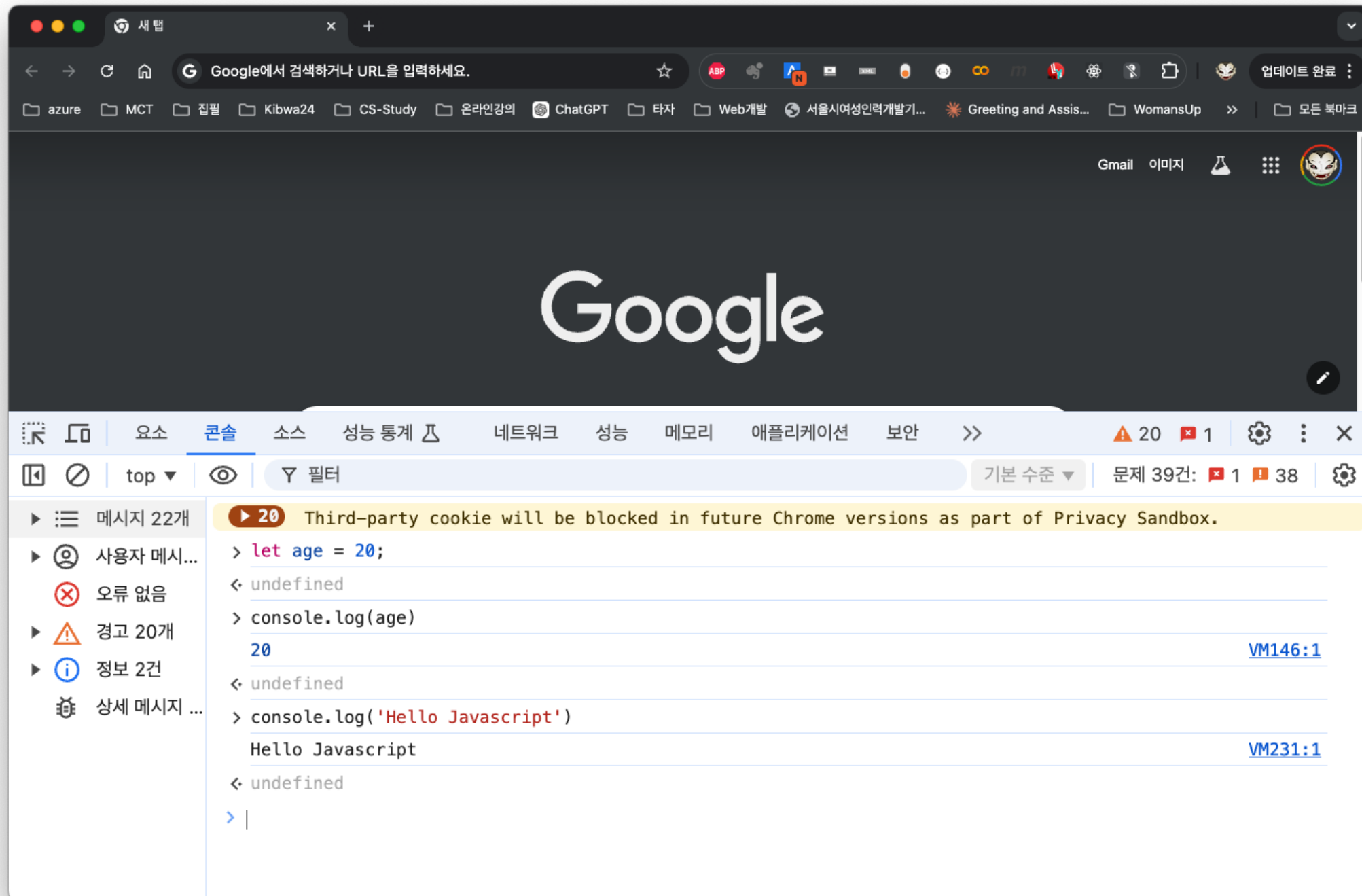
ECMA International에서 표준 제정

브라우저별 엔진

					
렌더링 엔진	Blink	Blink	Webkit	Gecko	Blink
JS 엔진	V8	V8	JavaScript Core	Spider Monkey	V8

개발환경설정

Chrome 개발자도구



Chrome의 REPL 환경



- JavaScript Runtime 환경
- Ryan Dahl이 2009년에 출시
- Chrome의 JavaScript 엔진인 V8엔진 사용
- 다른 JavaScript Runtime 환경으로 디노(<https://deno.land>), 번(<https://bun.sh>) 등이 있음

nodejs.org



Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

[Download Node.js \(LTS\)](#)

Downloads Node.js **v20.16.0**¹ with long-term support. Node.js can also be installed via [package managers](#).

Want new features sooner? Get **Node.js v22.6.0**¹ instead.

[Create an HTTP Server](#) [Write Tests](#) [Read and Has](#)

```
1 // server.mjs
2 import { createServer } from 'node:h
3
4 const server = createServer((req, re
5   res.writeHead(200, { 'Content-Type
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server local
10 server.listen(3000, '127.0.0.1', ()
11   console.log('Listening on 127.0.0.
12 });
13
14 // run with `node server.mjs`
```

JavaScript

Copy to clipboard

Learn more what Node.js is able to offer with our [Learning materials](#).



Visual Studio Code

- 마이크로소프트가 개발한 소스 코드 편집기
- 윈도우, Mac OS, 리눅스 등 다양한 플랫폼에서 사용 가능
- 다양한 프로그래밍 언어를 지원하며 각 언어와 함께 사용할 수 있는 편리한 기능들을 제공
- 플러그인을 통해 새로운 확장 기능 추가 가능

https://code.visualstudio.com

 Visual Studio Code Docs Updates Blog API Extensions FAQ

 Search Docs

Download

[Version 1.92](#) is now available! Read about the new features and fixes from July.

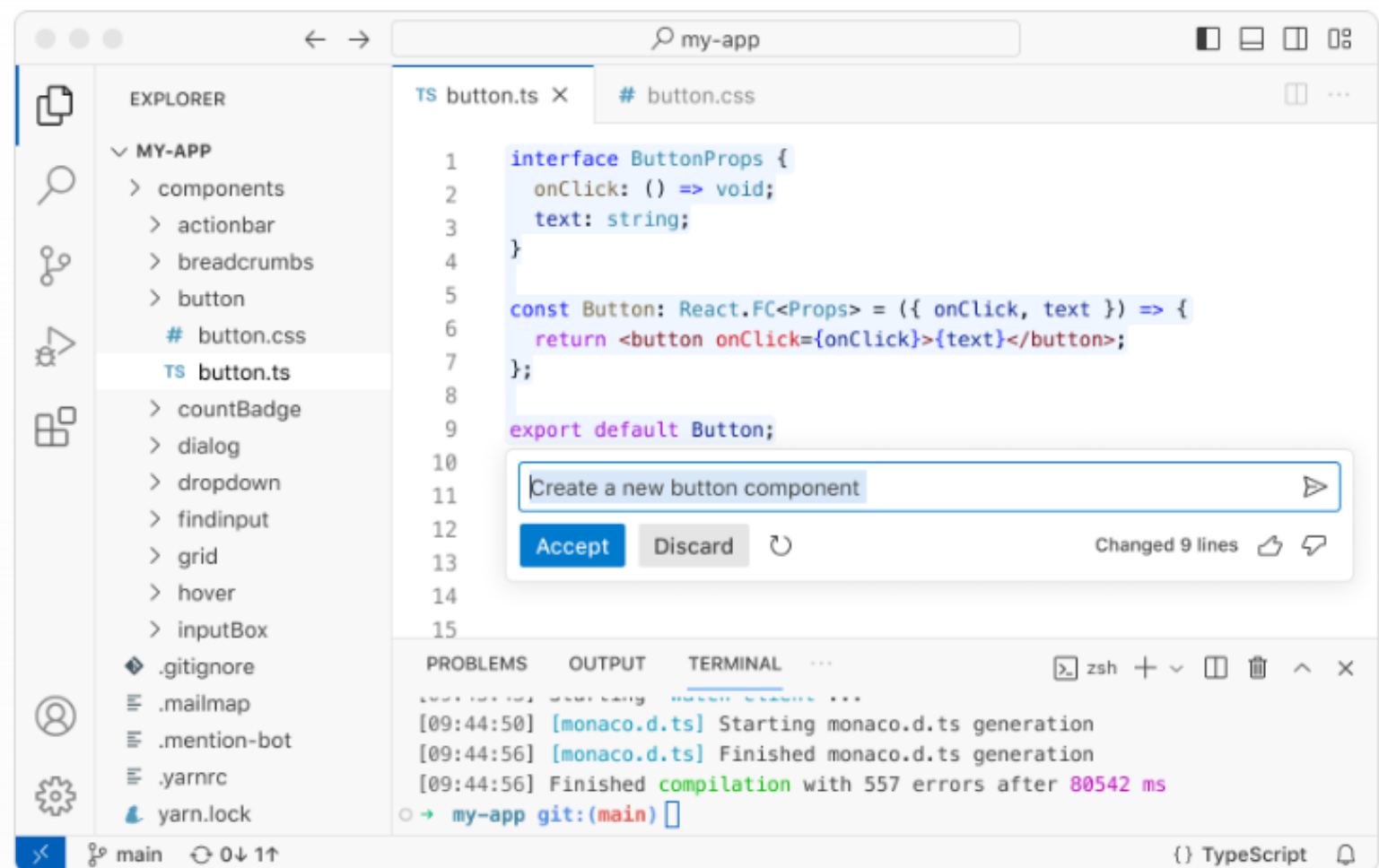
Free. Built on open source. Runs everywhere.

Code Editing. Redefined.

Download for macOS

[Web](#), [Insiders edition](#), or [other platforms](#)

By using VS Code, you agree to its [license](#) and [privacy statement](#).



Javascript 기본

상수와 변수

상수와 변수

- 상수와 변수는 값(데이터)을 저장하기 위한 메모리 공간
- 값을 저장하기 위해서 값을 저장할 공간을 선언해야함
- 상수(Constant) : 값을 한번 저장하면 바꿀 수 없음
- 변수(Variable) : 값을 저장한 후에도 필요 시 바꿀 수 있음

상수(Constant)

- `const age = 20;`
- `const` 으로 시작
- age는 상수명
- age라는 상수에 20이라는 값을 저장
- age는 상수로 선언했기 때문에 값을 변경할 수 없음

변수(Variable)

- `let grade = 3;`
- `let year;` //변수는 선언만 하는것도 가능(undefined)
- `year = 3;`
 - grade, year는 변수명
 - `year`라는 변수에 3이라는 값을 저장
- `year = 4;`
 - year는 변수로 선언했기 때문에 값 변경이 가능

console.log()

- 변수의 내용을 출력하는 함수
- `console.log(grade);`
- `console.log(grade, year);`
- `console.log('나이는 ' + age + '살이고 학년은 ' + grade + '학년이다');`
- template literal
 - `console.log(` 나이는 ${age}살이고 학년은 ${grade}학년이다.`);`

변수, 상수 작명규칙

- 동일한 스코프에서는 같은 이름으로 중복 선언할 수 없다(유일해야 한다).
- 예약어(let, const, if, switch 등)는 식별자로 사용할 수 없다.
- 식별자에는 문자, 숫자, _, \$를 사용할 수 있으며, 첫 글자에는 숫자를 사용할 수 없다.

```
let 😊 = "하하하"
```

이런것도 가능하지만 사용하지는 말자!

- 변수명과 상수명은 저장하는 데이터의 의미가 드러나게 작성한다.(studentName)
- 일반 변수명은 소문자로 시작하는 camelCase를 권장한다(scoreMath, studentName)
- 상수값은 필요에 따라 UPPER_SNAKE_CASE를 사용할 수 있다.

문장(Statement)

- `let age = 20;`
- `console.log(age);`
- javascript에서는 세미콜론(;) 을 사용해서 문장을 구분
- javascript 엔진이 세미콜론을 자동 삽입

주석(Comment)

- 코드에서 설명 또는 기록을 위해 코드가 아닌 내용을 추가할 때 사용
- // - 한 줄 주석, 주석으로 사용하고 싶은 문장의 앞에 사용

```
//이것은 한줄 주석입니다.  
const age = 20; //나이 설정
```

- /*~*/ - 여러줄 주석, 슬래쉬(/)와 애스터리크(*)를 사용

```
/* 여러줄 주석 문장1  
   여러줄 주석 문장2 */
```

```
//여러줄 주석 문장1  
//여러줄 주석 문장2
```

데이터형(Data Type)

데이터형(Data Type)

- 원시타입(Primitive data type)
 - number: 숫자
 - string: 문자열
 - boolean: true, false
 - null: 값이 없음
 - undefined: 정의되지 않음
 - symbol: 고유하고 변경 불가능한 값
- 객체타입(Object type)

number

- `let integer = 123; //정수`
- `let negative = -123; // 음수`
- `let double = 1.23; // 실수`
- `let binary = 0b10101010; // 2진수`
- `let octal = 0o157; // 8진수`
- `let hex = 0x7a3f; // 16진수`
- `let inf = 1/0 //Infinity`
- `let minf = -1/0 //-Infinity`
- `let nan = 1/'hello' //Nan(Not a Number)`

string

```
let greetings = '안녕하세요';  
greetings = "안녕하세요";  
greetings = "'안녕하세요'";  
greetings = '안녕하세요';  
const name = 'BTS';  
greetings = `안녕하세요 ${name}`; //Template Literal
```

boolean

```
let isFree = true;  
let isActivated = false;  
let isOn = true;  
console.log(isActivated);
```

연산자

산술연산자

연산자	사용예	의미
-	$-a$	값의 부호를 변경
+	$a + b$	두개의 값을 더함
-	$a - b$	연산자 앞의 값에서 뒤의 값을 뺌
*	$a * b$	두개의 값을 곱함
/	a / b	연산자 앞의 값에서 뒤의 값으로 나눔
%	$a \% b$	연산자 앞의 값을 뒤의 값으로 나눈 나머지 값

산술연산자 실습

```
let num1 = 5;
```

```
let num2 = 3;
```

```
console.log(num1); // 5
```

```
console.log(num2); // 3
```

```
console.log(-num1); // -5
```

```
console.log(num1 + num2); // 8
```

```
console.log(num1 - num2); // 2
```

```
console.log(num1 * num2); // 15
```

```
console.log(num1 / num2); // 1.666
```

```
console.log(num1 % num2); // 2
```

비교연산자

연산자	사용예	의미
<	$a < b$	a가 b보다 작으면 true, 그렇지 않으면 false
>	$a > b$	a가 b보다 크면 true, 그렇지 않으면 false
<=	$a \leq b$	a가 b보다 작거나 같으면 true, 그렇지 않으면 false
>=	$a \geq b$	a가 b보다 크거나 같으면 true, 그렇지 않으면 false
==	$a == b$	a가 b의 값이 같으면 true, 같지 않으면 false
===	$a === b$	a가 b와 값과 타입이 같으면 true, 같지 않으면 false
!=	$a != b$	a가 b와 값이 같지 않으면 true, 같으면 false
!==	$a !== b$	a가 b와 값과 타입이 같지 않으면 true, 같으면 false

비교연산자

```
console.log(123 == 123);
```

```
console.log(123 == '123');
```

```
console.log(123 === '123');
```

```
console.log(123 === 123);
```

```
console.log(123 != 123);
```

```
console.log(123 != '123');
```

```
console.log(123 !== '123');
```

```
console.log(123 !== 123);
```

논리연산자

연산자	사용예	의미
!(NOT)	!a	a가 true이면 false, false 이면 true 반환
&&(AND)	a && b	a와 b 모두 true일때 true, 하나라도 false 이면 false
(OR)	a b	a와 b 중 하나라도 true일때 true, 둘다 false 이면 false

비교, 논리연산자 실습

```
let num1 = 5;
```

```
let num2 = 3;
```

```
let num3 = 2;
```

```
console.log(num1 > num2 && num2 > num3); // true
```

```
console.log(num1 == num2 && num1 > num2); // false
```

```
console.log(num1 == num2 || num1 > num2); // true
```

```
console.log(num1 - num2 > num2 - num3 && num2 == 0); // false
```

```
console.log(num1 + num2 > num3 || num3 > 0); // true
```

연산자 우선순위 : 산술 → 비교 → && → ||

대입연산자

연산자	사용예	의미
=	a = 3	변수 a에 3을 대입
+=	a += 1	a = a + 1
-=	a -= 3	a = a - 3
*=	a *= 3	a = a * 3
/=	a /= 3	a = a / 3
%=	a %= 3	a = a % 3

증감연산자

연산자	사용예	의미
++	num++	필요한 작업 후 1증가
++	++num	1증가 후 필요한 작업
--	num--	필요한 작업 후 1감소
--	--num	1감소 후 필요한 작업

대입, 증감연산자 실습

```
let num = 0;
console.log(num);
num++; // a = a + 1;
console.log(num);
num--; // a = a - 1;
console.log(num);
// num++ 필요한 작업 후 값을 증가(후치)
// ++num 값을 증가후 필요한 작업수행(전치)
num = 1;
console.log(num++);
num = 1;
console.log(++num);
let num1 = num++;
console.log(num1);
console.log(num);
```


삼항 조건 연산자

연산자	사용예	의미
? :	조건 ? A : B	조건이 참이면 A, 거짓이면 B를 반환

```
const age = 20;  
const result = age > 19 ? '당신은 성인입니다.' : '당신은 미성년자입니다.';  
console.log(result);
```

연산자 우선순위

```
let a = 2;  
let b = 3;  
let result = a + (b * 4) / 5;  
console.log(result);  
result = ((a + b) * 4) / 5;  
console.log(result);
```

```
result = a++ + b * 4;  
console.log(result);  
console.log(a)
```

() → 단항연산자(+, -, !, ++, --) → *, /, % → +, -

조건문

if 조건문

```
const adult = 19;  
let age = 15;  
  
if (age < adult) {  
    console.log('당신은 미성년자네요');  
}
```

```
if (age < adult) {  
    console.log('당신은 미성년자네요');  
} else {  
    console.log('당신은 성인이네요');  
}
```

if 조건문

```
let gender = 'male';

if (age < adult) {
  if (gender === 'male') {
    console.log('당신은 미성년 남성이네요');
  } else {
    console.log('당신은 미성년 여성이네요');
  }
}
```

중첩 if

if 조건문

```
let isLoggedIn = true
let token = 0

if (isLoggedIn && token) {
  console.log('로그인 상태입니다');
} else if (isLoggedIn && !token) {
  console.log('토큰이 없습니다');
} else {
  console.log('로그인이 필요합니다');
}
```

&& 조건

if 조건문

```
let age = 15;
let isMember = false;

if (age < 18 || isMember) {
  console.log('할인 대상입니다');
} else {
  console.log('할인 대상이 아닙니다');
}
```

|| 조건

다중선택문(if else ~ if)

```
const browser = '크롬';  
let browserName;  
  
if (browser == 'Edge') {  
    browserName = '엣지';  
} else if (browser == 'Safari') {  
    browserName = '사파리';  
} else if (browser == '크롬') {  
    browserName = '크롬';  
} else {  
    browserName = '알려지지 않은 브라우저';  
}  
console.log(`브라우저명은 ${browserName}`);
```


다중선택문(switch case)

```
let menu = 2;  
switch (menu) {  
  case 1:  
    console.log('아메리카노');  
    break;  
  case 2:  
    console.log('카페라떼');  
    break;  
  case 3:  
    console.log('카푸치노');  
    break;  
  default:  
    console.log('없는 메뉴입니다');  
}
```

다중선택문(switch case)

```
let browser1 = 'Chrome';
let browserName1;
switch (browser1) {
  case 'Edge':
    browserName1 = '엣지'; break;
  case 'FF':
    browserName1 = '파이어폭스'; break;
  case 'Chrome':
    browserName1 = '크롬'; break;
  case 'Opera':
    browserName1 = '오페라'; break;
  case 'Safari':
    browserName1 = '사파리'; break;
  default:
    browserName1 = '알려지지 않은 브라우저';
}
console.log(`브라우저명은 ${browserName1}`);
```

Truthy – Falsy

```
console.log(`true is ${Boolean(true)}`);  
console.log(`false is ${!!false}`);  
console.log(`0 is ${Boolean(0)}`);  
console.log(`-0 is ${Boolean(-0)}`);  
console.log(`1 is ${Boolean(1)}`);  
console.log(`-1 is ${Boolean(-1)}`);  
console.log(`' ' is ${Boolean(' ')}`);  
console.log(`'0' is ${Boolean('0')}`);  
console.log(`'false' is ${Boolean('false')}`);  
console.log(`null is ${Boolean(null)}`);  
console.log(`undefined is ${Boolean(undefined)}`);  
console.log(`NaN is ${Boolean(NaN)}`);  
console.log(`[] is ${Boolean([])}`);  
console.log(`{} is ${Boolean({})}`);
```

Quiz-학점 구하기

주어진 score에 따라서 학점(A,B~F)을 출력하는
조건문(if~else if) 를 이용해서 작성하기

Quiz-요일 출력하기

변수 day는 요일을 나타내는 숫자(0:일~6:토)를 가진다.
day값에 따라 요일을 출력하는 프로그램을 작성하시오.

반복문(Loop)

for 구문

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

```
for (let i = 0; i < 5; i++) {  
  for (let j = 0; j < 5; j++) {  
    console.log(i, j);  
  }  
}
```

중첩 for

for 구문

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

```
for (let i = 0; i < 5; i++) {  
  for (let j = 0; j < 5; j++) {  
    console.log(i, j);  
  }  
}
```

중첩 for

Quiz-구구단 출력

구구단을 이중 for루프를 이용해서 출력하시오.

while 구문

```
let age = 0;

while (age < 5) {
  console.log(`${age}살입니다.`);
  age += 1;
}
```

while 구문

```
let age = 0;
let num = 0;
while (age < 5) {
  while (num < 10) {
    console.log(`${age}살입니다.`);
    num += 1;
  }
  num = 0;
  age += 1;
}
```

do ~ while 구문

```
let num = 0;
```

```
do {  
    num = num + 1;  
    console.log(num);  
} while (num < 10);
```

1번 실행 후 조건 검사, 한 번 이상 실행해야 할때 사용

break, continue

```
for (let i = 0; i < 10; i++) {  
  if (i == 2) {  
    break;  
  }  
  console.log(`${i} was executed!`);  
}
```

```
for (let i = 0; i < 10; i++) {  
  if (i === 2) {  
    continue;  
  }  
  console.log(`${i} was executed!`);  
}
```

break, continue

```
let age = 0;
while (age < 5) {
  age += 1;
  if (age === 2) continue;
  console.log(`${age}살입니다.`);
}
```

```
let age = 0;
while (age < 5) {
  age += 1;
  if (age === 2) break;
  console.log(`${age}살입니다.`);
}
```

Quiz-홀수 출력

for루프, continue, %(나머지 연산자)를 이용해서
0~10 사이의 홀수를 출력하라.

Function

함수 (Function)

- 프로그램의 기본적인 구성요소
- 특정 작업을 수행하는 코드의 묶음
- 하나의 기능(동작)을 수행하는 코드 블록
- 하나의 프로그램은 여러 개의 함수들이 모여서 구성
- 재 사용성 및 가독성의 증대, 유지 관리가 용이

함수 정의

```
function sayHello() { //함수정의
    console.log('Hello');
}
sayHello(); //함수호출
```

```
function sayHello1(name) { //매개변수
    console.log(`Hello ${name}`);
}
sayHello1('World!');
sayHello1(1);
```

Quiz-함수 정의 실습

구구단의 단을 매개변수로 전달받아
출력하는 함수를 정의하고 호출하시오.

함수 정의-반환값

```
function sayHello2(name) {  
    return `Hello ${name}`;  
    //함수가 처리한 연산의 결과값을 반환하고 함수 실행을 종료  
}
```

```
let greeting = sayHello2('RM');  
console.log(greeting);  
console.log(sayHello2('morning'));
```

```
function add(num1, num2) {  
    return num1 + num2;  
}
```

```
let result = add(5, 4);  
console.log(result);
```

함수 정의-반환값

```
function sayHello3(name) {  
  console.log('함수시작');  
  return;  
  console.log('실행안됨');  
}  
greeting = sayHello3('RM');  
console.log(greeting);
```

return문이 없는 함수도 undefined를 반환

Quiz-함수 정의 실습

점수(score)를 매개 변수로 전달받아
학점(A,B~F)을 반환하는 함수를 정의하고 여러번 호출하시오.

Quiz-함수 정의 실습

두 수와 연산자(string형)를 입력받아서
사칙연산 후 결과를 반환하는 calc함수를 만드시오.
단 예외 사항은 고려하지 않는다.

함수의 할당

```
const sayHello4 = sayHello;  
sayHello4();  
sayHello();
```

```
const calculator = calc;  
console.log(calculator(2, '+', 3));
```


함수형 매개변수

```
function sayHello(name) {  
    console.log(`Hello ${name}`);  
}  
  
function sayHi(name) {  
    console.log(`Hi ${name}`);  
}  
  
function greet(name, fn) {  
    //doSomething  
    console.log(name);  
    fn(name);  
}  
  
greet('간부', sayHi);  
greet('선생님', sayHello);
```

Quiz-함수 정의 실습

사칙연산을 담당하는 각각의 함수를 정의하고
정의된 함수들을 매개변수로 전달받아서
처리하는 calc2 함수를 정의하고 호출하시오.

다양한 함수 정의 방식

//함수 선언문

```
function add1(n1, n2) {  
    return n1 + n2;  
}  
console.log(add1(1, 2));
```

//함수 표현식

```
let add2 = function (n1, n2) {  
    return n1 + n2;  
};  
console.log(add2(1, 2));
```

// 화살표 함수

```
let add3 = (n1, n2) => {  
    return n1 + n2;  
};  
console.log(add3(1, 2));
```

화살표함수(Arrow Function)

// 기본형태

```
let fn = (매개변수) => {  
  //doSomething  
};
```

// 매개변수가 1개일때 괄호 생략가능

```
let square = x => {  
  return x * x;  
};
```

// 매개변수가 0개일때 괄호 생략 불가

```
const sayHi = () => {  
  console.log('Hi');  
};
```

화살표함수(Arrow Function)

// 함수바디가 한 줄이면 자동 return, {}같이 생략해야함

```
const add = (a, b) => a + b;
```

// 반환형이 객체일 경우 ()로 감싸야 함

```
const getUser = () => ({ name: 'Son' });
```

매개변수로써 화살표함수

```
function greet1(name, callback) {  
  return callback(name);  
}
```

```
let result;  
result = greet1('선생님', (name) => {  
  return `Hello ${name}`;  
});  
console.log(result);
```

```
//return 생략  
result = greet1('간부', (name) => `Hi ${name}`);  
console.log(result);
```

Quiz-Callback 실습

calc2 함수를 호출할때 함수 표현식과 화살표 함수를 사용해서 호출하시오.

Object

Object

- 데이터와 기능을 한데 묶어 관리할 수 있는 중요한 자료형
- 객체는 키(Key)와 값(Value)의 쌍으로 구성
- 키(Key)와 값(Value)을 Object의 속성이라고 함
- 함수(Function)를 속성으로 가질 수 있음
- 객체내에 정의된 함수를 메서드(Method)라고 함

객체 리터럴

```
const human = {  
  name: '라일랜드', //객체의 속성  
  age: 30,  
};  
//객체 내부의 속성에 접근할때는 '.', 또는 ["속성명"]을 사용  
human.age = 20;  
human['name'] = '그레이스';  
console.log(human.name, human['age']);
```

객체 리터럴-Method

```
const eridian = {  
  name: '로키', //객체의 속성  
  age: 30,  
  greet: function () { //객체의 Method  
    //this는 현재의 object 자신을 의미함(호출방식에따라 바뀜)  
    console.log(`안녕 나는 ${this.name}야!`);  
  },  
  goodbye() { //Method 축약표현  
    console.log('goodbye');  
  },  
};  
eridian.greet();  
eridian.goodbye();
```

객체에 속성 추가 삭제

```
human.job = 'teacher';  
console.log(human.job);
```

```
human.info = function () {  
    console.log(`이름은 ${this.name}이고  
                직업은 ${this.job}이다.`);  
};  
human.info();
```

```
delete human.age;  
console.log(human.age);
```

생성자 함수

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function () {  
    console.log(`안녕 나는 ${this.name}야!`);  
  };  
}
```

```
const person1 = new Person('양관식', 20);  
const person2 = new Person('오애순', 20);  
console.log(person1.name, person1.age);  
person2.greet();
```

class 사용

```
class Person1 {  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
    greet() {  
        console.log('Hello, ' + this.name);  
    }  
}  
  
// person3은 Person1 클래스의 인스턴스  
  
const person3 = new Person1('양금명', 20);  
console.log(person3.age, person3.name);  
person3.greet();
```

class 사용-private

```
class Person2 {  
  #idNo; //private ES2022 부터지원  
  constructor(name, age, idNo) {  
    this.name = name;  
    this.age = age;  
    this.#idNo = idNo;  
  }  
  getIDNo() {  
    console.log(`idNo:${this.#idNo}`);  
  }  
}  
  
const person4 = new Person2('양금명', 20, '123456');  
person4.getIDNo();  
// console.log(person4.age, person4.#idNo); //외부에서 접근불가
```

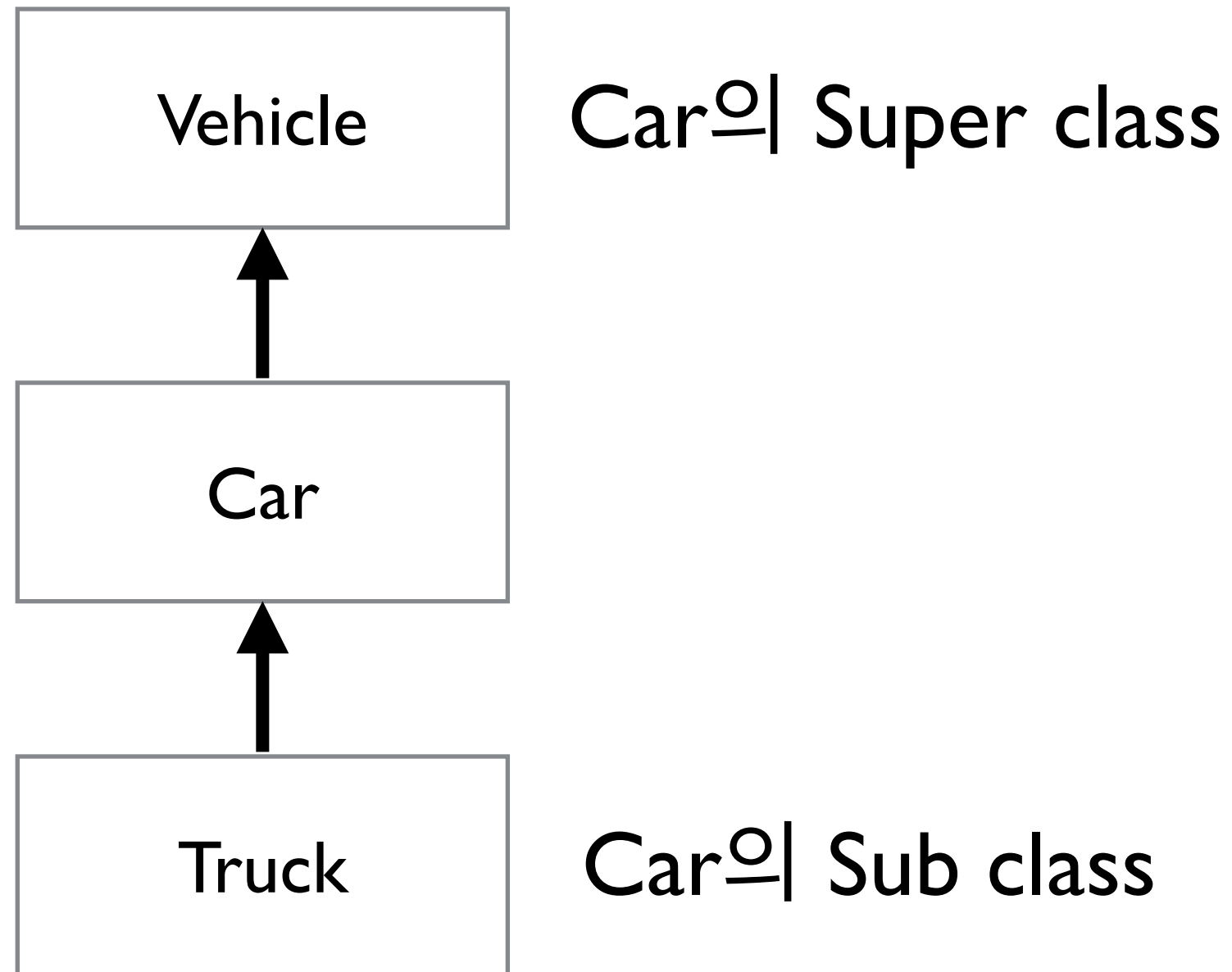

클래스 정의 실습

- 운송수단을 의미하는 Vehicle 클래스를 정의하라.
- 속도를 관리하는 speed라는 속성을 정의하라.
- speed를 10 증가하는 메소드를 정의하라.
- speed를 10 감소하는 메소드를 정의하라.
- 현재속도를 표시하는 info 메소드를 정의하라.

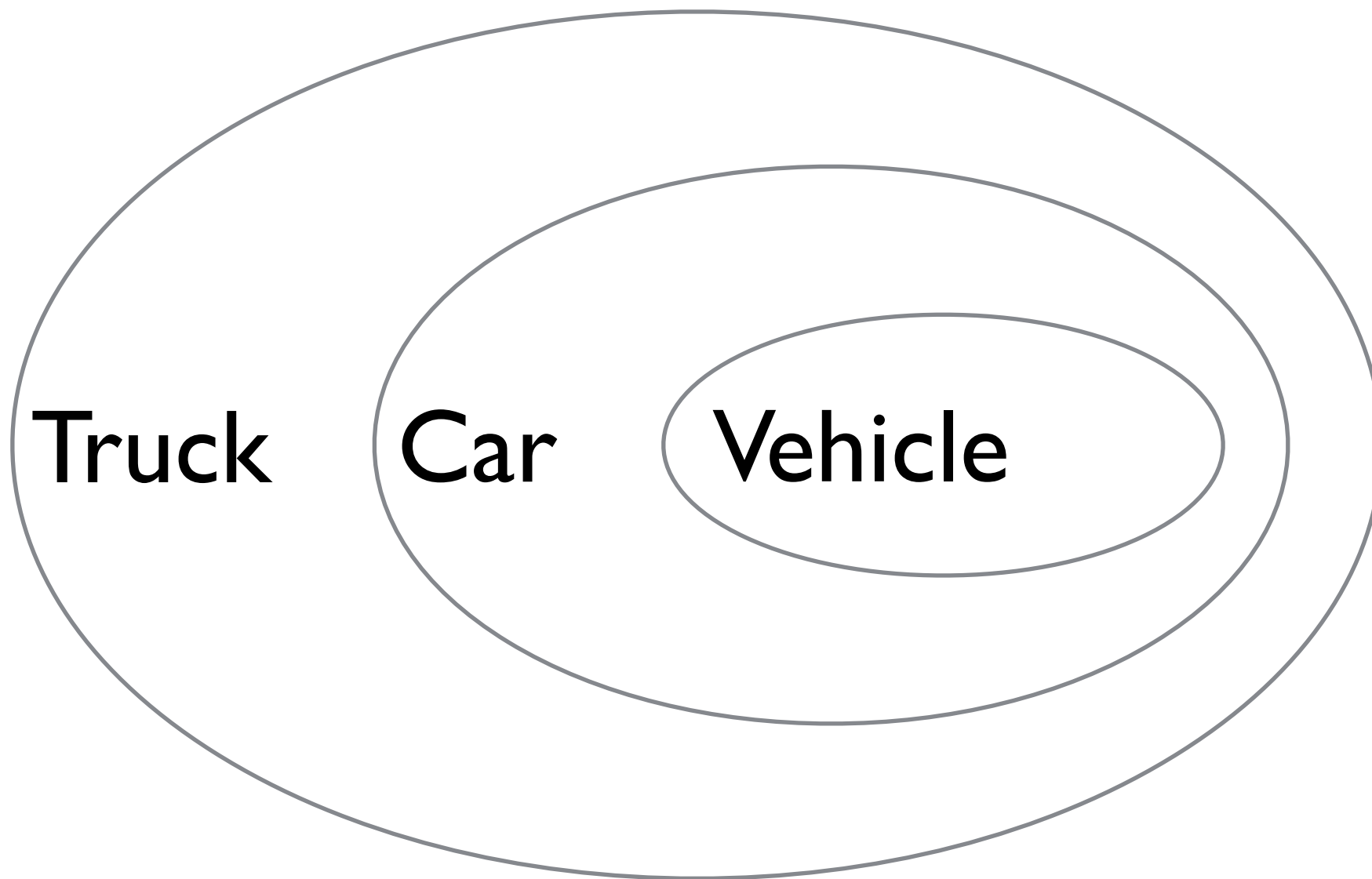
클래스 정의 실습

```
class Vehicle {  
  constructor(speed) {  
    this.speed = speed;  
  }  
  speedUp() {  
    this.speed += 10;  
  }  
  speedDn(){  
    this.speed -= 10;  
  };  
}  
const vehicle = new Vehicle(0);  
vehicle.speedUp();  
console.log(vehicle.speed);  
vehicle.speedDn();  
console.log(vehicle.speed);
```

클래스의 상속



클래스의 상속



상속의 구현

```
class Car extends Vehicle {  
  constructor(speed, wheels, seats) {  
    super(speed);  
    this.wheels = wheels;  
    this.seats = seats;  
  }  
  drive() { console.log(`현재 속도는 ${this.speed}로 운행`); }  
}  
  
const car = new Car(100, 4, 4);  
car.speedUp();  
console.log(car.speed);  
car.drive();
```

클래스 상속 실습

- Car class를 상속받아서 Truck class를 정의하라.
- 적재량을 관리하는 loadage 정수형 변수를 정의하고 초기값 0으로 설정
- 'load'를 화면에 인쇄하는 load 메소드를 정의하라.
- 'unLoad'를 화면에 인쇄하는 unLoad 메소드를 정의하라.

Collection Type

Collection Type(집단자료형)

- 서로 연관이 있는 데이터끼리 모아서 관리
- 데이터를 그룹단위로 묶을 수 있으므로 다량에 데이터 다룰때 편리함
- 배열(Array) : 일련번호로 구분되는 순서에 따라 데이터가 정렬된 목록 형태의 자료형
- 집합(Set) : 중복되지 않은 데이터들이 모인 집합 형태의 자료형
- 맵(Map) : 키(key)-값(value)로 연관된 데이터들이 모인 자료형

Array 선언과 초기화

```
const numbers = [1, 2, 3, 4, 5];  
console.log(numbers[0]) //index는 0부터 시작
```

```
let array = new Array(2); //size  
console.log(array);
```

```
array = new Array(1, 2, 3, 4, 5);  
console.log(array);
```


item 갯수, 수정, 추가, 삭제

//item의 갯수

```
console.log(numbers.length);
```

```
numbers[3] = -4;
```

```
numbers[5] = 6; //추가
```

```
console.log(numbers.length, numbers);
```

```
numbers[8] = 7;
```

```
console.log(numbers.length, numbers);
```

//item 삭제

```
delete numbers[1];
```

```
console.log(numbers.length, numbers);
```

Array 활용

```
let bts1 = ['진', '슈가', '제이홉', 'RM'];  
console.log(bts1);  
let bts2 = new Array('지민', '뷔', '정국');  
console.log(bts2);  
console.log(bts1.indexOf('슈가'));  
console.log(bts2.indexOf('슈가'));  
  
console.log(bts1.includes('RM'));  
console.log(bts2.includes('지민'));
```

Array 활용

// 추가

```
bts1.push('박보검'); // 맨뒤에 추가
```

```
console.log(bts1);
```

```
console.log(bts1.length);
```

```
bts1.unshift('이도현'); // 맨앞에 추가
```

```
console.log(bts1);
```

// 삭제

```
let first = bts1.shift(); // 첫번째 아이템 삭제
```

```
console.log(bts1);
```

```
console.log(first);
```

```
let last = bts1.pop(); // 마지막 아이템 삭제
```

```
console.log(last);
```

```
console.log(bts1);
```

Array 활용

//특정한 index에서 count만큼 삭제 splice(index, count)

```
const deleted = bts1.splice(1, 1);
```

```
console.log(bts1);
```

```
console.log(deleted);
```

//특정한 index에서 추가 splice(index, 0, item, item,...)

```
bts1.splice(1, 0, '박보검', '이도현');
```

```
console.log(bts1);
```

Array 활용

//일부분으로 새로운 배열을 만듦

```
let bts3 = bts1.slice(0, 2);  
console.log(bts3);  
console.log(bts1);
```

```
bts3 = bts1.slice(-1);  
console.log(bts3);  
bts3 = bts1.slice(-3);  
console.log(bts3);
```

Array 활용

// 배열 합치기

```
const bts = bts1.concat(bts2);  
console.log(bts);
```

// 역순 배열, bts가 바뀜

```
const rbts = bts.reverse();  
console.log(rbts);
```

// 배열을 문자열로 합하기

```
let members = bts.join(', ');  
console.log(members);
```

Array 활용

```
const fruits = ['Banana', 'Orange', 'Apple', 'Mango'];  
fruits.sort();  
console.log(fruits);  
// Output: ["Apple", "Banana", "Mango", "Orange"]
```

```
const numbers = [40, 100, 1, 5, 25, 10];  
numbers.sort();  
console.log(numbers);  
// Output: [1, 10, 100, 25, 40, 5] (incorrect for numbers)
```


Array 활용

```
// 오름차순 정렬
```

```
// n1 - n2 > 0 이면 자리 바꿈
```

```
numbers.sort(function (n1, n2){ return n1 - n2 });  
console.log(numbers);
```

```
// Output: [1, 5, 10, 25, 40, 100]
```

```
numbers.sort((n1, n2) => n2 - n1);  
console.log(numbers);
```

```
// Output: [ 100, 40, 25, 10, 5, 1 ]
```


Array 활용

```
const items = [  
  { name: 'Apple', price: 50 },  
  { name: 'Banana', price: 10 },  
  { name: 'Mango', price: 30 },  
];  
//item의 가격을 기준으로 정렬  
items.sort((a, b) => a.price - b.price);  
console.log(items);  
  
items.sort((a, b) => b.price - a.price);  
console.log(items);
```

Array 순회

```
const bts = ['RM', '진', '슈가', '제이홉', '지민', '뷔', '정국'];  
//for loop 사용  
for (let i = 0; i < bts.length; i++) {  
    console.log(bts[i]);  
}  
//for~of 사용  
for (let member of bts) {  
    console.log(member);  
}  
//forEach 사용  
bts.forEach((member, index, array) => {  
    console.log(`전체 ${array.length}명중에 ${index}번째  
        멤버 ${member}입니다.`);  
});
```

Quiz-Array 활용

list array의 item 중에 bts 멤버만 골라 배열을 만들고 하나의 문자열로 만들어서 출력하시오.

```
const list = ['슈가', '차은우', '박서준', '이도현', '제이홉', '주우재', '지민'];  
const bts = ['진', '슈가', '제이홉', 'RM', '지민', '뷔', '정국'];
```

map() 함수

```
const nums = [1, 2, 3, 4, 5];
```

```
// 고차원 함수인 map은 함수를 인자로 받음
```

```
const doubled = nums.map((num) => {  
  return num * 2;  
});
```

```
console.log(doubled);
```

```
// 출력: [2, 4, 6, 8, 10]
```

filter() 함수

```
const nums = [1, 2, 3, 4, 5];
```

```
const evenNumbers = nums.filter((num) => {  
  //결과가 true이면 통과  
  return num % 2 === 0;  
})
```

```
console.log(evenNumbers);
```

```
// 출력: [2, 4]
```

Quiz-Array 활용

list array의 item 중에 bts 멤버만 filter를 사용해서 골라 배열을 만들고 하나의 문자열로 만들어서 출력하시오.

```
const list = ['슈가', '차은우', '박서준', '이도현', '제이홉', '주우재', '지민'];  
const bts = ['진', '슈가', '제이홉', 'RM', '지민', '뷔', '정국'];
```

reduce() 함수

```
const nums = [1, 2, 3, 4, 5];
```

```
const sum = nums.reduce((accumulator, currentValue) => {  
    return accumulator + currentValue;  
}, 0); // 초기값
```

```
console.log(sum); // 출력: 15
```


Quiz-고차원함수 활용

1~10까지의 숫자 중에 3의 배수만 골라서
2배한 후 합을 구하시오

Set 선언과 초기화

```
const set = new Set([1, 2, 3, 4, 5]);  
console.log(set);
```

```
// 사이즈 확인
```

```
console.log(set.size);
```

```
//item 추가
```

```
set.add(6);
```

```
console.log(set);
```

```
set.add(6);
```

```
console.log(set);
```

Set 활용

// 존재유무

```
console.log(set.has(3));  
console.log(set.has(6));
```

// 삭제

```
set.delete(6);  
console.log(set);
```

// 전부 삭제

```
set.clear();  
console.log(set);
```

Set 순회

```
// 순회
// forEach
set.forEach((item) => console.log(item));

// for ~ of
for (const value of set.values()) {
  console.log(value);
}
```

Map 선언과 초기화

```
const map = new Map([[ 'nick', '슈가'], [ 'group', 'bts'],  
                     [ 'gender', '남'],]);  
console.log(map);
```

```
// 사이즈 확인
```

```
console.log(map.size);
```

```
// 존재하는지 확인
```

```
console.log(map.has('name'));
```

```
console.log(map.has('group'));
```

Map 활용

```
console.log(map.get('name'));  
console.log(map.get('group'));
```

// 추가

```
map.set('nation', 'korea');  
console.log(map);
```

// 삭제

```
map.delete('gender');  
console.log(map);
```

Map 순회

// 순회

```
map.forEach((value, key) => console.log(key, value));  
console.log(map.keys());  
console.log(map.values());  
console.log(map.entries());
```

Map과 Object의 차이점

특성	Object	Map
키 유형	문자열 또는 심볼	모든 자료형
삽입된 순서	일부 보장 (숫자 키는 정렬될 수 있음)	삽입된 순서가 항상 보장됨
키-값 쌍 관리 방식	프로토타입 속성 충돌 가능	순수한 키-값 쌍 관리
크기 확인 방법	<code>Object.keys().length</code> 사용	<code>size</code> 프로퍼티 사용
성능	많은 양의 데이터에서는 다소 느림	더 나은 성능 (특히 대규모 데이터 처리 시)
반복	<code>for...in</code> , <code>Object.keys()</code> 등	<code>for...of</code> , <code>forEach()</code> 사용

Math

```
console.log(Math.abs(-10));  
console.log(Math.ceil(1.4));  
console.log(Math.floor(1.4));  
console.log(Math.round(1.49));  
console.log(Math.round(1.5));  
console.log(Math.trunc(1.565465));  
console.log(Math.random() * 10);  
console.log(Math.floor(Math.random() * 10) + 1);
```


Quiz-로또발생기

랜덤 발생함수 `Math.random()` 함수를 사용해서 로또 발생기를 만드시오. 정렬은 하지않아도 됨

※ `Math.random()`함수는 0이상 1미만의 난수를 생성

단축평가, 객체분해,
문자열조작

단축 평가

(Short-circuit Evaluation)

```
console.log(true && 'hello'); // "hello"  
console.log(false && 'hello'); // false
```

```
console.log(true || 'hello'); // true  
console.log(false || 'hello'); // "hello"
```

단축 평가

// 사용자가 이름을 입력하지 않았을 때(null 또는 undefined) 기본값 할당

```
let userName = '' ; // Falsy
```

```
let displayName = userName || 'Guest' ;
```

```
console.log(displayName); // "Guest"
```

// 값이 존재할 경우

```
userName = 'Wizard' ;
```

```
displayName = userName || 'Guest' ;
```

```
console.log(displayName); // "Wizard"
```

단축 평가

```
let isLoggedIn = true;
let userProfile = { name: 'Alice' };

// 사용자가 로그인했을 때만 이름을 출력
isLoggedIn && console.log(userProfile.name); // "Alice"

// 로그인하지 않았다면 아무 일도 일어나지 않음
isLoggedIn = false;
isLoggedIn && console.log(userProfile.name); // (출력 없음)
```

Null 병합 연산자-??

```
const name1 = null ?? 'Guest';  
console.log(name1);
```

```
const name2 = undefined ?? "Guest";  
console.log(name2);
```

```
const name3 = "RM" ?? "Guest";  
console.log(name3);
```

```
const name4 = "" || "Guest";  
console.log(name4);
```

```
const name5 = "" ?? "Guest";  
console.log(name5);
```

객체 분해(Object Destructuring)

```
const user = {  
  name: 'RM',  
  age: 30,  
};  
// 기존 방식  
const name1 = user.name;  
const age2 = user.age;  
  
// 객체 분해 방식  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```


객체 분해 - alias

```
const user1 = {  
  name: 'RM',  
  age: 30,  
};
```

```
const { name: userName, age: userAge } = user1;
```

```
console.log(userName);  
console.log(userAge);
```


객체 분해-객체 매개 변수

```
function userInfo(user) {  
    console.log(`${user.name} - ${user.age}`);  
}
```

```
userInfo({ name: 'RM', age: 30 });
```

```
function printUser({ name, age }) {  
    console.log(`${name} - ${age}`);  
}
```

```
printUser({ name: 'RM', age: 30 });
```

Spread 연산자 ...

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1];  
console.log(arr2);
```

```
const person = { name: 'RM', age: 20 };  
const copy = { ...person };  
console.log(copy);
```

```
const user = { name: '손흥민', age: 20 };  
const newUser = { ...user, age: 21 };  
console.log(newUser);
```

함수 정의-Rest 매개변수

```
function sum(...nums) {  
    console.log(nums);  
}  
sum(1, 2, 3, 4, 5);
```

```
function sum1(num1, num2, ...nums) {  
    console.log(num1);  
    console.log(num2);  
    console.log(nums);  
}  
sum1(1, 2, 3, 4, 5);
```

Rest 매개변수는 반드시 마지막에 와야 한다

문자열 조작

```
const str = 'JavaScript';
```

```
console.log(str.length);  
console.log(str.toUpperCase());  
console.log(str.toLowerCase());  
console.log(str.includes('script'));  
console.log(str.includes('Java'));  
console.log(str.startsWith('java'));  
console.log(str.endsWith('Script'));  
console.log(str.indexOf('a'));  
console.log(str.indexOf('z'));  
console.log(str.slice(0, 4));  
console.log(str.slice(4));  
console.log(str.replace('Script', 'script'));
```

문자열 조작

```
const str1 = 'a,b,c';  
const arr = str1.split(',');  
console.log(arr);  
const str2 = '  hello  ';  
console.log(str2.trim());  
const str3 = 'ha';  
console.log(str3.repeat(3));  
const str4 = 'hello';  
console.log(str4.charAt(1));
```

예외처리

try-catch-finally

```
try {  
    // 실행할 코드  
} catch (error) {  
    // 에러 발생 시 실행  
}
```

```
try {  
    console.log(a); // a는 정의되지 않음  
} catch (error) {  
    console.log('에러 발생!');  
}
```

```
try {  
    console.log('실행');  
} catch (e) {  
    console.log('에러');  
} finally {  
    console.log('무조건 실행');  
}
```


에러 throw

```
try {  
    throw new Error('문제 발생');  
} catch (error) {  
    console.log(error.message);  
}  
  
function checkAge(age) {  
    if (age < 18) {  
        throw new Error('미성년자');  
    }  
    return '통과';  
}  
  
try {  
    checkAge(15);  
} catch (error) {  
    console.error(error);  
}
```


Javascript Module

Javascript Module

- 모듈이란 코드를 파일 단위로 나누고 필요한 것만 가져다 쓰는 구조

- 코드 분리 (유지보수)
- 재 사용성 증가
- 협업 용이

- import 방식(ES Module)

- ES6 이후 Javascript 표준 방식

- require 방식 (CommonJS)

- Javascript를 브라우저 외부에서 사용 시 모듈화 작업을 할수 있게 CommonJS라는 워킹 그룹이 제안한 방식
- Node.js는 모듈 명세 1.0을 준수, Node.js 사용 시 기본

named export/import

```
//math.js
```

```
export const add = (a, b) => a + b;
```

```
export const multiply = (a, b) => a * b;
```

```
// export { add, multiply };
```

```
//app.js
```

```
import { add, multiply } from './math.js';
```

```
console.log(add(2, 3)); // 5
```

```
console.log(multiply(2, 3)); // 6
```

default export/import

//greet.js

```
export default function greet(name) {  
  return `${name}님, 안녕하세요.`;  
}
```

//app.js

```
import greet from './greet.js';  
  
console.log(greet('로키'));
```

named+default

```
//user.js
```

```
export const age = 20;
```

```
export default function getName() {  
  return '그레이스';  
}
```

```
//app.js
```

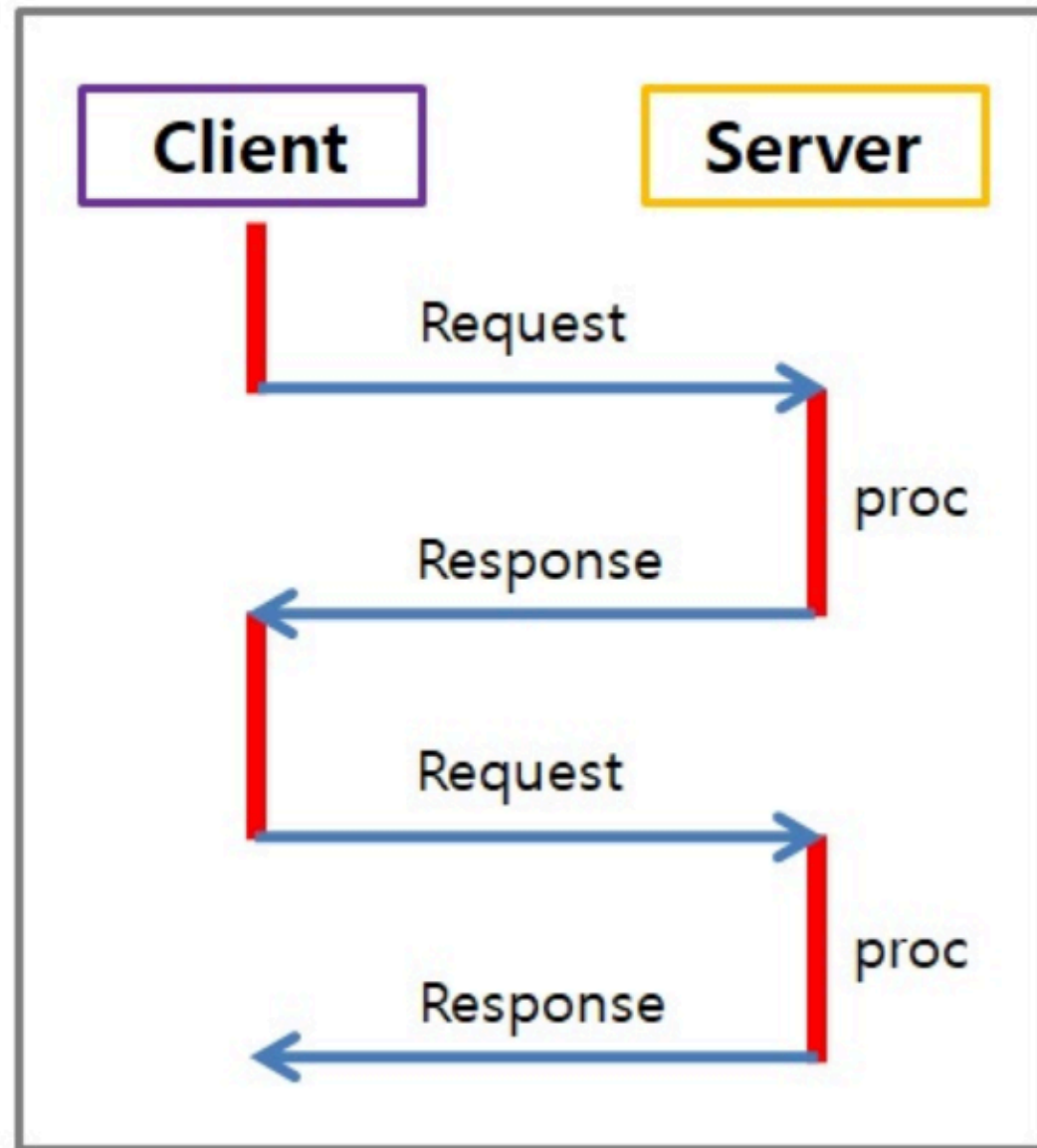
```
import getName, { age } from './user.js';
```

```
console.log(getName());  
console.log(age);
```

비동기 프로그래밍

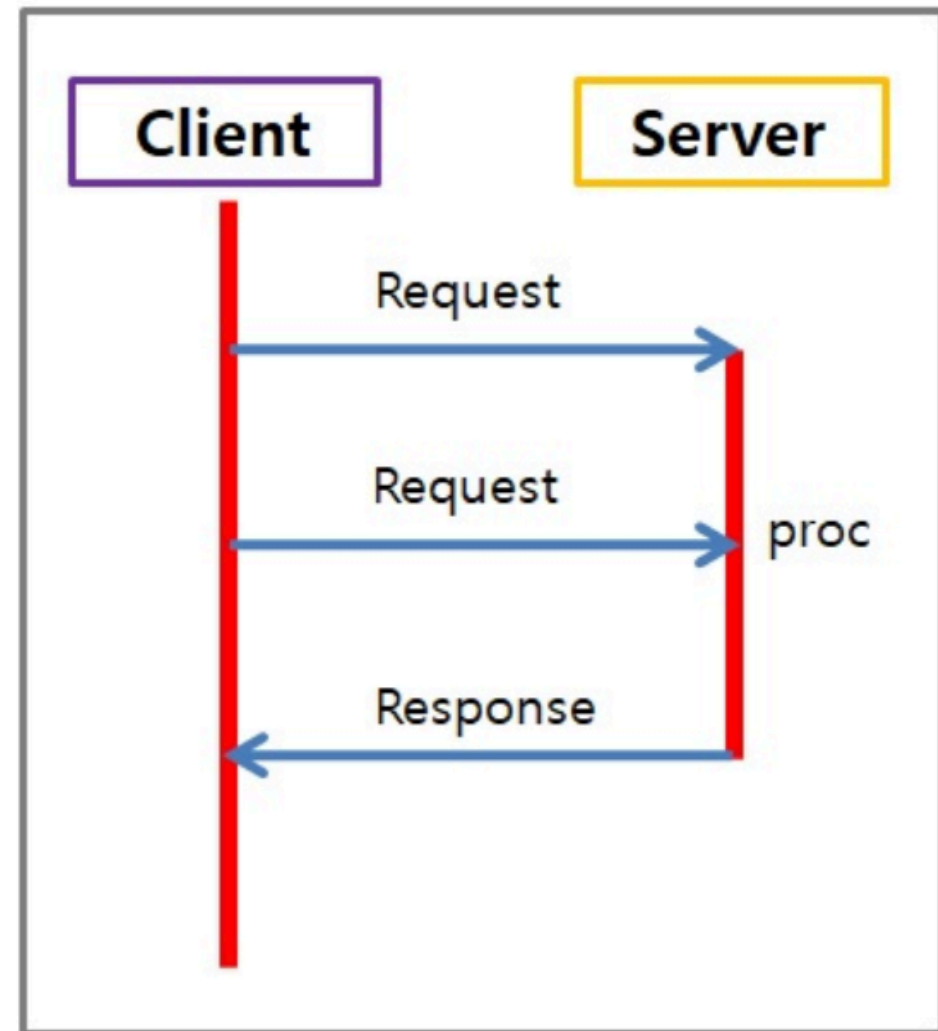
동기 프로그래밍

```
console.log('1번');  
console.log('2번');  
console.log('3번');
```



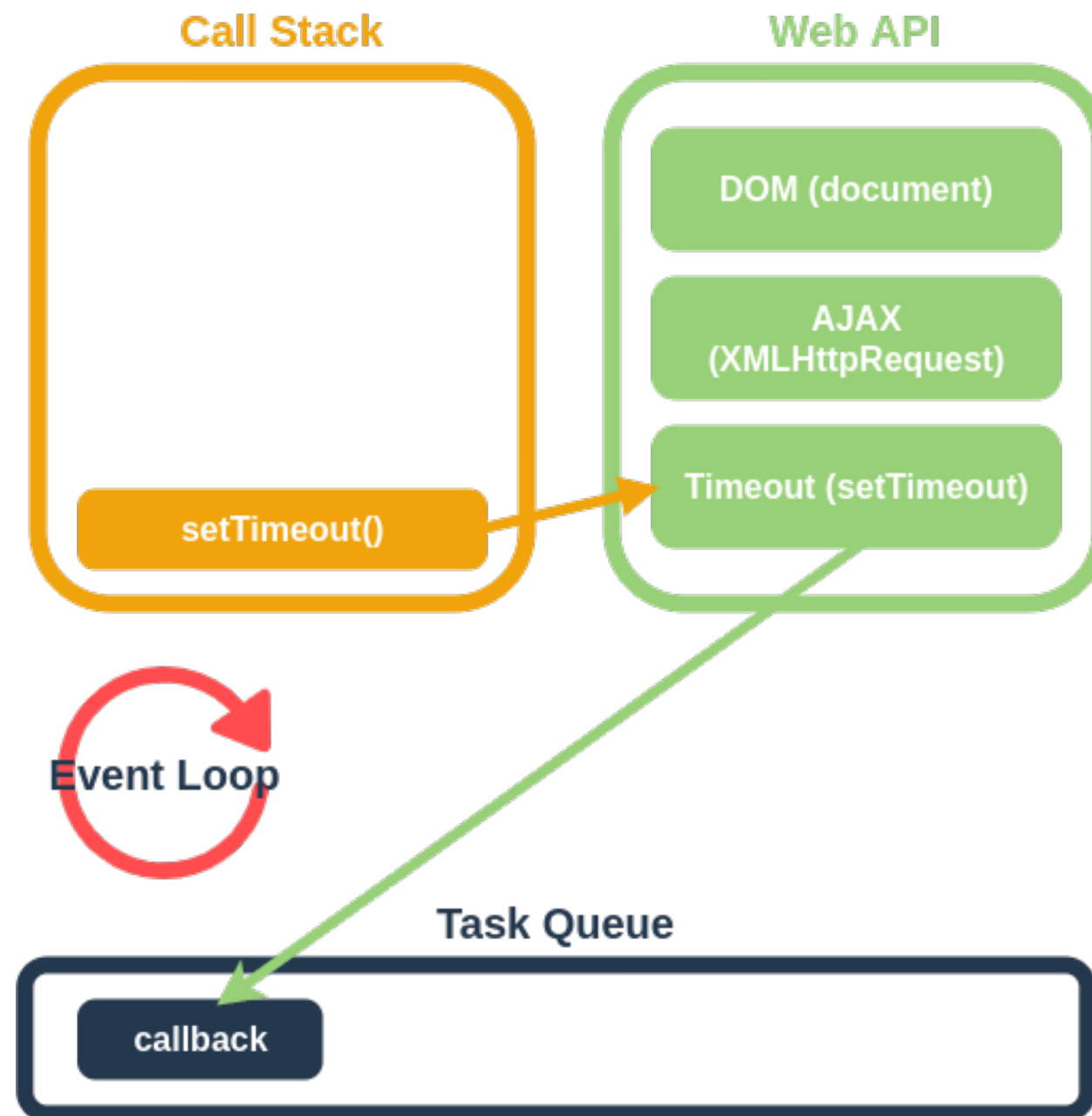
비동기 프로그래밍

```
console.log('1번');  
setTimeout(() => {  
  console.log('2번');  
}, 1000);  
console.log('3번');
```



- setTimeout
 - 특정 작업을 일정 시간이 지난 후에 실행하도록 예약하는 비동기 함수
 - n 밀리초(ms) 후에 전달된 콜백을 실행 시킵니다.

비동기 프로그래밍-Web API (Event Loop, Task Queue)



비동기 프로그래밍

```
function asyncTest(name, callback) {  
  console.log('타이머 시작');  
  setTimeout(() => {  
    callback(name);  
  }, 3000); // 3초 후에 데이터를 가져온다고 가정  
}
```

```
function doOtherthing() {  
  for (let i = 0; i < 300; i++) {  
    console.log(`${i}번째 처리`);  
  }  
}  
  
asyncTest('뷔', sayHello);  
doOtherthing();
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
const DB = [];
```

```
function save2DB(user, callback) {  
  DB.push(user);  
  console.log(`${user.name}님 데이터베이스에 저장 완료되었습니다.`);  
  return callback(user);  
}
```

```
function sendEmail(user, callback) {  
  console.log(`${user.email}으로 이메일이 전송 완료되었습니다.`);  
  return callback(user);  
}
```

```
function getResult(user) {  
  return `${user.name}님 회원가입에 성공했습니다.`;  
}
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
function register(user) {  
  return save2DB(user, (user) => {  
    return sendEmail(user, (user) => {  
      return getResult(user);  
    });  
  });  
}
```

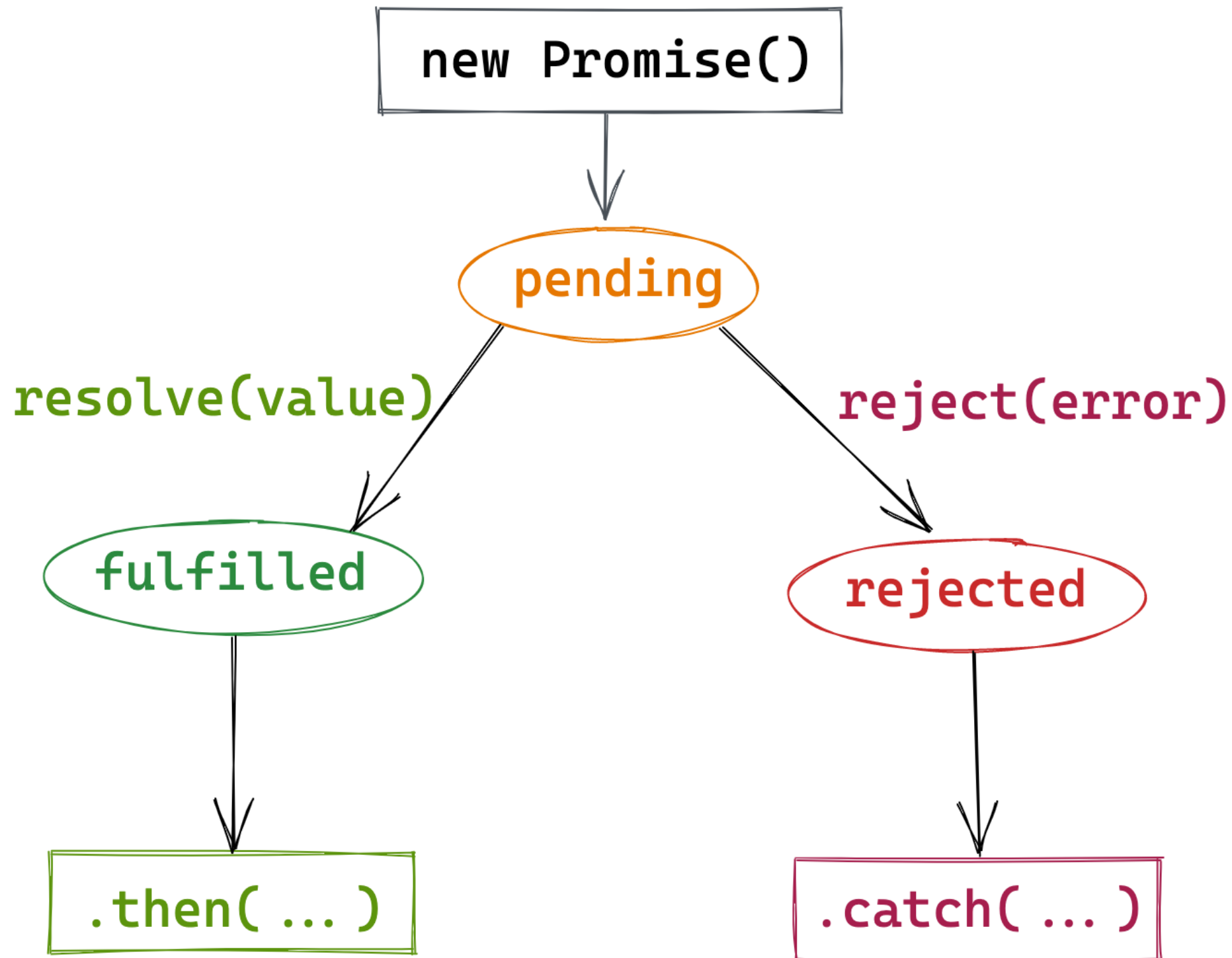
```
const result =  
  register({ name: '손흥민', email: 'son@naver.com' });  
console.log(result);
```



Promise

- 콜백의 단점을 보완하기 위해 만들어진 비동기 처리 객체
- 비동기 처리 시점을 명확히 표현 가능
- 연속된 비동기 처리 작업을 처리하기 간편
- Promise는 3개의 상태를 가짐
 - pending: 아직 실행되지 않은 초기 상태
 - fulfilled: 연산이 성공적으로 완료됨
 - rejected: 연산이 실패함

Promise



Promise

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve('작업 성공!');  
  } else {  
    reject('작업 실패!');  
  }  
});  
promise  
  .then((result) => {  
    console.log('성공 결과:', result);  
  })  
  .catch((error) => {  
    console.error('실패 결과:', error);  
  });
```


Promise

```
const p = new Promise((resolve) => {  
  resolve(10);  
  console.log('1. Promise 실행');  
});  
  
console.log('2. 코드 계속 실행');  
  
p.then((num) => {  
  console.log('3. then 실행:', num);  
});
```


Promise-체이닝

```
const p1 = new Promise((resolve) => {  
  const result = 10;  
  resolve(result);  
});
```

//반환값:11 => resolve(11)을 호출하는 Promise를 생성

```
const p2 = p1.then((num) => num + 1);
```

```
p2.then((num) => console.log(num));
```

```
p1.then((num) => num + 1).then((num) => console.log(num));
```

즉시 성공 Promise

```
new Promise((resolve) => {  
    resolve(10);  
});  
Promise.resolve(10);  
//함수가 항상 Promise를 반환하게 만들기  
function getData() {  
    return Promise.resolve('데이터');  
}  
getData().then(console.log);  
//Promise 체이닝을 사용  
Promise.resolve(1)  
    .then((n) => n + 1)  
    .then(console.log);
```

즉시 실패 Promise

```
Promise.reject('에러').catch(console.log);
```

```
function checkAge(age) {  
  if (age < 18) {  
    return Promise.reject('미성년자');  
  }  
  return Promise.resolve('통과');  
}
```

```
checkAge(15).catch(console.log);
```

Promise - 회원가입

```
const DB = [];  
  
function saveDB(user) {  
  const oldDBLength = DB.length;  
  DB.push(user);  
  console.log(`${user.username} 저장 완료되었습니다.`);  
  return new Promise((resolve, reject) => {  
    if (DB.length > oldDBLength) {  
      resolve(user);  
    } else {  
      reject(new Error('저장에 실패했습니다.!' ));  
    }  
  });  
}
```

Promise - 회원가입

```
function sendEmail(user) {  
  console.log(`${user.email}으로 이메일을 발송했습니다.`);  
  return new Promise((resolve) => {  
    resolve(user);  
  });  
}
```

```
function getResult(user) {  
  return new Promise((resolve) => {  
    resolve(`${user.username}님 등록 성공했습니다.`);  
  });  
}
```

Promise - 회원가입

```
function registerByPromise(user) {  
  const result = saveDB(user)  
    .then(sendEmail)  
    .then(getResult)  
    .catch((error) => new Error(error));  
  return result;  
}
```

```
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
const result = registerByPromise(myUser);  
result.then(console.log);
```

Promise - 영화정보

```
const url =  
  'http://raw.githubusercontent.com/wapj/jsbackend/main/  
movieinfo.json';  
  
fetch(url)  
  .then((response) => {  
    return response.json();  
  })  
  .catch(() => {  
    console.log('요청에 실패했습니다.');  })
```


Promise - 영화정보

=> 앞에서 계속

```
.then((data) => {  
    if (!data) {  
        throw new Error('데이터가 없습니다.');    }  
    if (!data.articleList || data.articleList.length === 0) {  
        throw new Error('데이터가 없습니다.');    }  
    return data.articleList;  
})  
.catch((error) => {  
    console.error('에러 발생:', error.message);  
})
```


Promise - 영화정보

=> 앞에서 계속

```
.then((articles) => {  
    return articles.map((article, idx) => {  
        return { title: article.title, rank: idx + 1 };  
    });  
})  
.then((results) => {  
    for (let movie of results) {  
        console.log(`[${movie.rank}위] ${movie.title}`);  
    }  
})  
.catch((err) => {  
    console.log('<<에러발생>>');  
    console.log(err);  
});
```

async/await

- Promise를 더 쉽게 다루기 위한 문법
- 비동기 코드를 동기 코드처럼 깔끔하게 작성 가능
- async function은 자동으로 Promise로 감싸서 반환
- await는 Promise가 해결될 때까지 대기
- await는 반드시 async 함수 안에서만 사용 가능
- await 오른쪽은 반드시 Promise

async/await

// async 키워드가 붙은 함수는 반환값을 Promise로 감싸서 반환

```
async function func1() {  
  return 'hello';  
}
```

```
func1().then(console.log);
```

```
function func2() {  
  return new Promise((resolve) => {  
    resolve('hello');  
  });  
}
```

```
func2().then(console.log);
```

async/await

```
async function func3() {  
  //await는 뒤에 오는 프로미스가 실행완료될때까지 기다림  
  //async함수에서만 사용가능  
  let name = await func1();  
  console.log(name);  
}  
  
func3();
```

async/await

```
function waitOneSecond(msg) {  
  return new Promise((resolve, _) => {  
    setTimeout(() => resolve(`${msg}`), 1000);  
  });  
}
```

```
async function countOneToTen() {  
  const nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  for (let x of nums) {  
    let result = await waitOneSecond(`${x + 1}초 대기 중...`);  
    console.log(result);  
  }  
  console.log('완료');  
}  
countOneToTen();
```

async/await - 회원가입

```
async function registerByAsync(user) {  
  try {  
    const savedUser = await saveDB(user);  
    const emailedUser = await sendEmail(savedUser);  
    const result = await getResult(emailedUser);  
    return result;  
  } catch (error) {  
    return new Error(error);  
  }  
}  
  
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
registerByAsync(myUser).then(console.log);
```

async/await - 영화정보

// 1. 데이터 가져오기(fetch)

```
async function fetchMovieData(url) {  
  const response = await fetch(url);  
  if (!response.ok) {  
    throw new Error('요청에 실패. 상태 코드: ' + response.status);  
  }  
  const data = await response.json();  
  return data;  
}
```

// 2. 데이터 검증

```
function validateMovieData(data) {  
  if (!data.articleList || data.articleList.length === 0) {  
    throw new Error('데이터가 없습니다. ');  
  }  
}
```


async/await - 영화정보

// 3. 영화 정보 추출

```
function extractMovieInfos(articleList) {  
  return articleList.map((article, idx) => ({  
    title: article.title,  
    rank: idx + 1,  
  }));  
}
```

// 4. 영화 출력

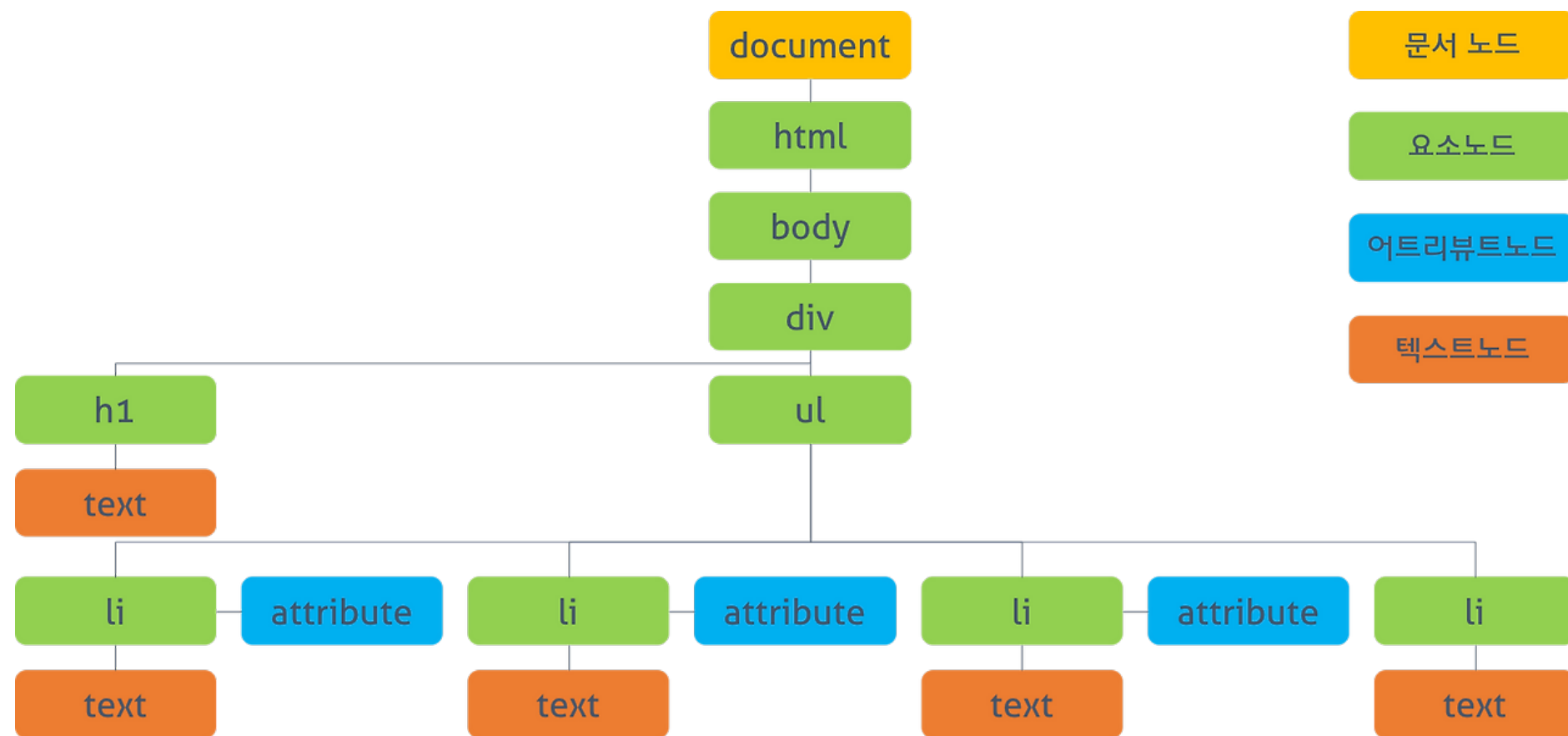
```
function displayMovies(movieInfos) {  
  for (const movie of movieInfos) {  
    console.log(`[${movie.rank}위] ${movie.title}`);  
  }  
}
```


async/await - 영화정보

```
const url = 'http://raw.githubusercontent.com/wapj/  
jsbackend/main/movieinfo.json';  
  
async function movies() {  
  try {  
    const data = await fetchMovieData(url);  
    validateMovieData(data);  
    const movieInfos = extractMovieInfos(data.articleList);  
    displayMovies(movieInfos);  
  } catch (err) {  
    console.error('에러 발생:', err.message);  
  }  
}  
  
movies();
```

DOM(문서 객체 모듈) 다루기

DOM(Document Object Model)



- DOM은 HTML 문서를 객체로 표현한 트리 구조 모델
- 브라우저는 DOM을 사용해 렌더링
- 자바스크립트는 이를 조작한다

요소 선택

```
<body>
  <h1 id="title">제목입니다</h1>
  <div class="box">box 클래스 요소</div>
  <div><p>div 바로 아래에 있는 p 태그</p></div>
  <script>
    const el1 = document.querySelector('#title');
    const el2 = document.querySelector('.box');
    const el3 = document.querySelector('div > p');
    console.log(el1);
    console.log(el2);
    console.log(el3);
  </script>
</body>
```

요소 조작 및 이벤트

```
<body>
  <h1 id="title">안녕하세요</h1>
  <button class="btn">변경</button>
  <script>
    const title = document.querySelector('#title');
    const btn = document.querySelector('.btn');
    btn.addEventListener('click', () => {
      title.textContent = 'querySelector 성공!';
    });
  </script>
</body>
```

다중 선택

```
<body>
  <ul>
    <li>1번</li>
    <li>2번</li>
    <li>3번</li>
  </ul>

  <script>
    const items = document.querySelectorAll('li');
    items.forEach((item) => {
      item.style.color = 'blue';
    });
  </script>
</body>
```

class 조작

```
<body>
  <div class="box">클릭</div>
  <style>
    .active {
      background: yellow;
    }
  </style>
  <script>
    const box = document.querySelector('.box');
    box.addEventListener('click', () => {
      box.classList.toggle('active');
    });
  </script>
</body>
```


요소 생성

```
<body>
  <ul class="list"></ul>
  <button class="add">추가</button>
  <script>
    const list = document.querySelector('.list');
    const btn = document.querySelector('.add');
    btn.addEventListener('click', () => {
      const li = document.createElement('li');
      li.textContent = '새 항목';
      list.appendChild(li);
    });
  </script>
</body>
```


입력처리

```
<body>
  <input class="input" placeholder="입력" />
  <button class="add">추가</button>
  <ul class="list"></ul>
  <script>
    const input = document.querySelector('.input');
    const btn = document.querySelector('.add');
    const list = document.querySelector('.list');
    btn.addEventListener('click', () => {
      const li = document.createElement('li');
      li.textContent = input.value;
      list.appendChild(li);
      input.value = '';
    });
  </script>
</body>
```

DOM 선택 메서드

```
<body>
  <h1 id="title">제목</h1>
  <div class="box">box1</div>
  <div class="box">box2</div>
  <p>문단1</p>
  <p>문단2</p>
  <input type="text" name="username" value="홍길동" />
  <input type="text" name="username" value="김철수" />
  <script>
    const el1 = document.getElementById('title');
    console.log('getElementById:', el1);
    const el2 = document.getElementsByClassName('box');
    console.log('getElementsByClassName:', el2);
    const el3 = document.getElementsByTagName('p');
    console.log('getElementsByTagName:', el3);
    const el4 = document.getElementsByName('username');
    console.log('getElementsByName:', el4);
  </script>
</body>
```

prototype

prototype

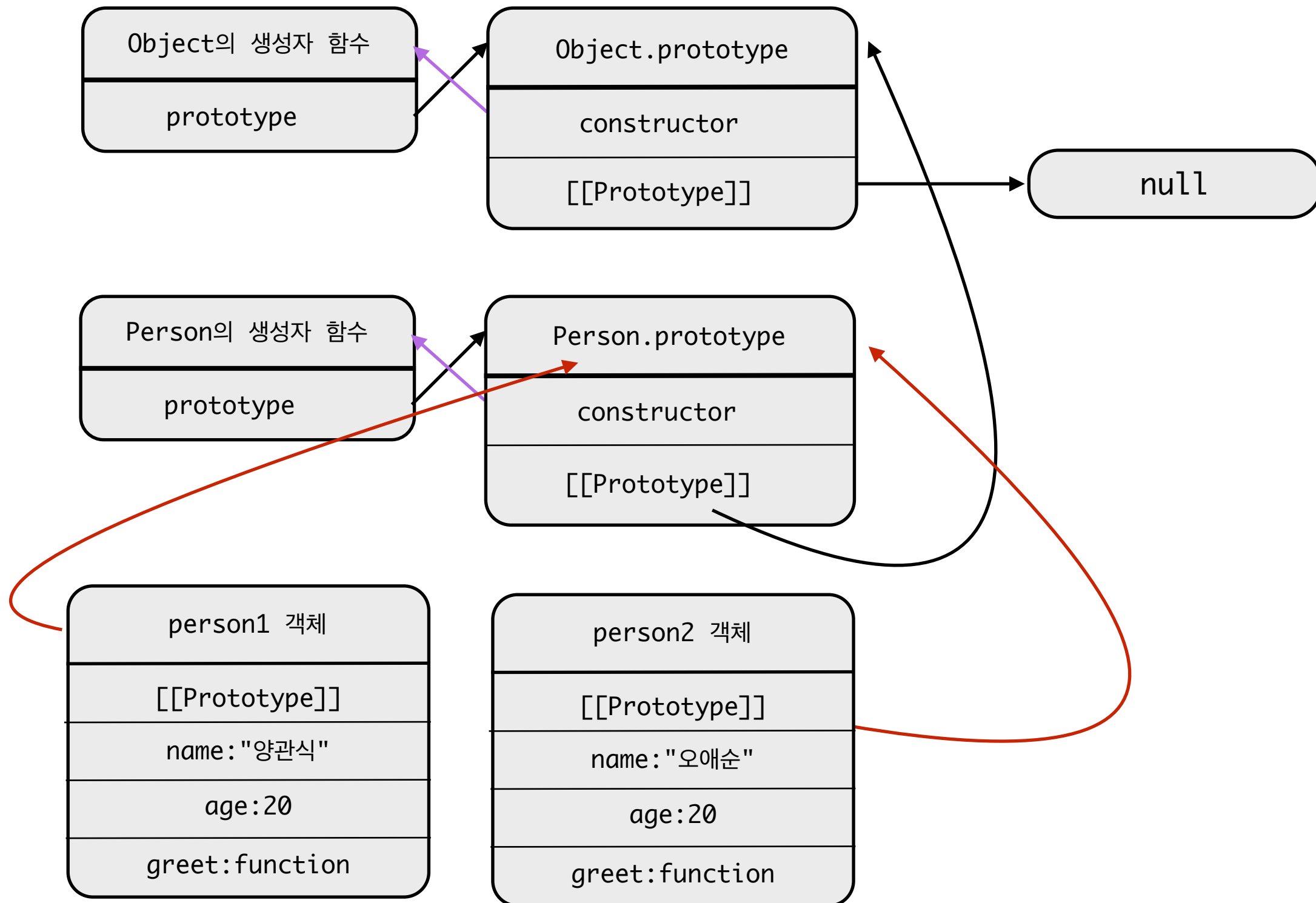
- prototype은 JavaScript의 상속을 구현하는 핵심 메커니즘
- prototype은 객체를 생성할 때 상속될 속성과 메서드를 담는 공간
- 객체에서 값을 찾을 때 prototype 체인을 따라 올라가며 탐색
- 객체는 `[[Prototype]]`를 통해 생성자의 prototype과 연결
- 모든 생성자 함수는 prototype 객체를 가진다

생성자 함수

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function () {  
    console.log(`안녕 나는 ${this.name}야!`);  
  };  
}
```

```
const person1 = new Person('양관식', 20);  
const person2 = new Person('오애순', 20);  
console.log(person1.name, person1.age);  
person2.greet();
```

prototype을 이용한 상속



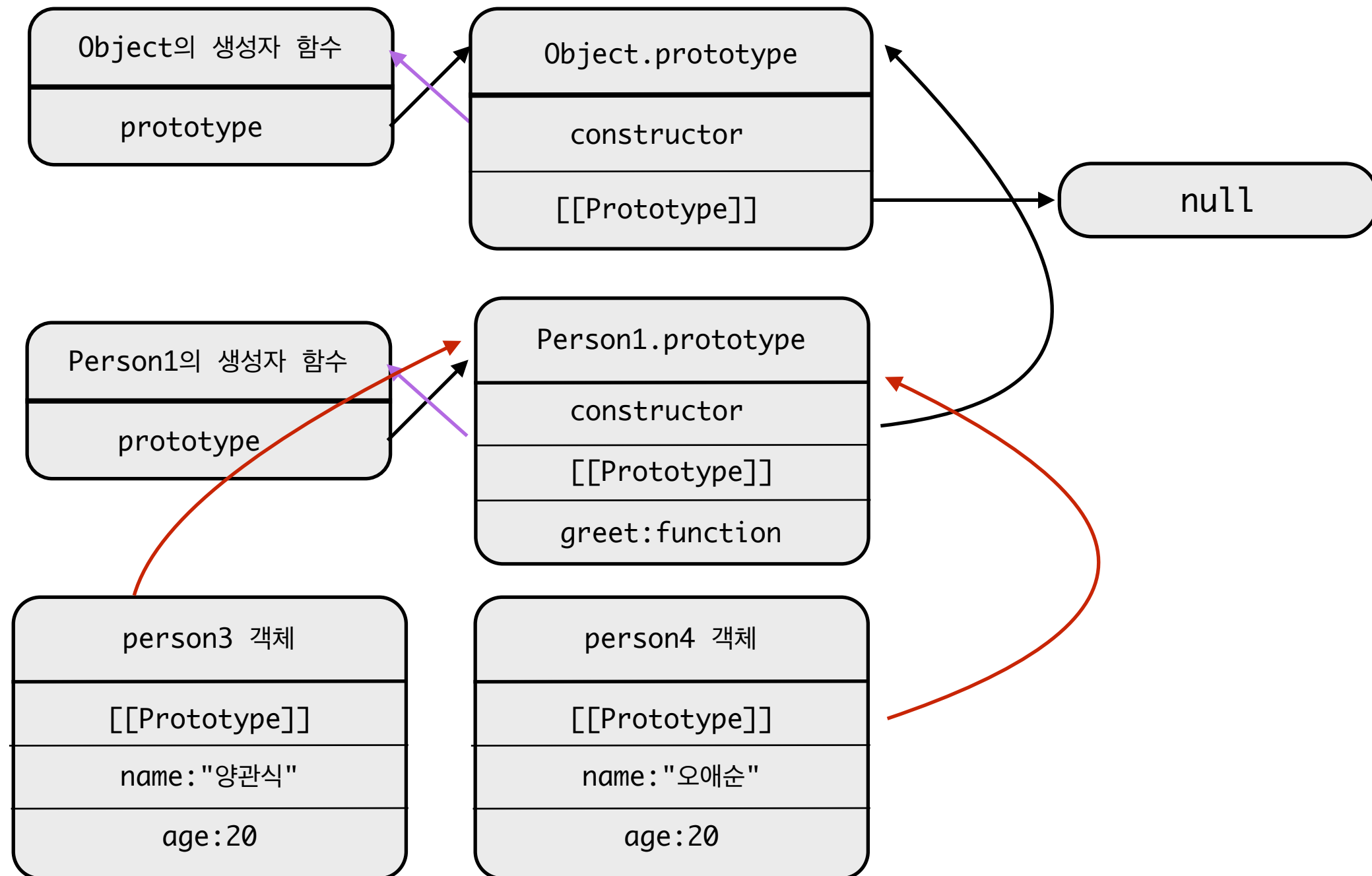
prototype 메서드

```
function Person1(name, age) {  
    this.name = name;  
    this.age = age;  
}  
Person1.prototype.greet = function () {  
    console.log(`안녕 나는 ${this.name}야!`);  
};
```

```
const person1 = new Person1('양관식', 20);  
const person2 = new Person1('오애순', 20);
```

```
console.log(person1.name, person1.age);  
person2.greet();
```

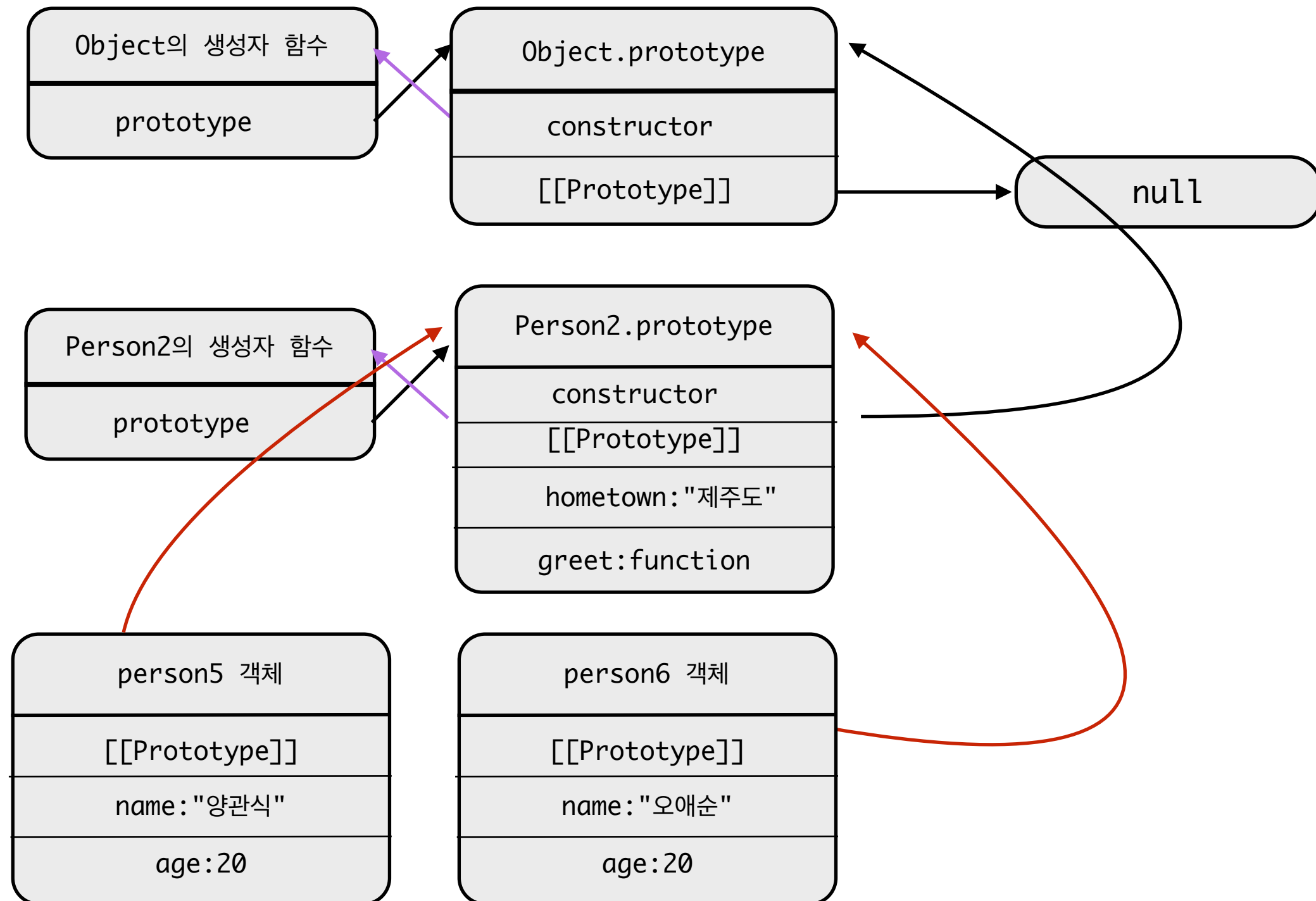

prototype에 메서드 추가



prototype 속성

```
function Person2(name, age) {  
  this.name = name;  
  this.age = age;  
}  
Person2.prototype.greet = function () {  
  console.log(`안녕 나는 ${this.name}야!`);  
};  
Person2.prototype.hometown = '제주도';  
const person5 = new Person2('양관식', 20);  
const person6 = new Person2('오애순', 20);  
console.log(person5.hometown, person6.hometown);  
Person2.prototype.hometown = '서울';  
console.log(person5.hometown, person6.hometown);
```

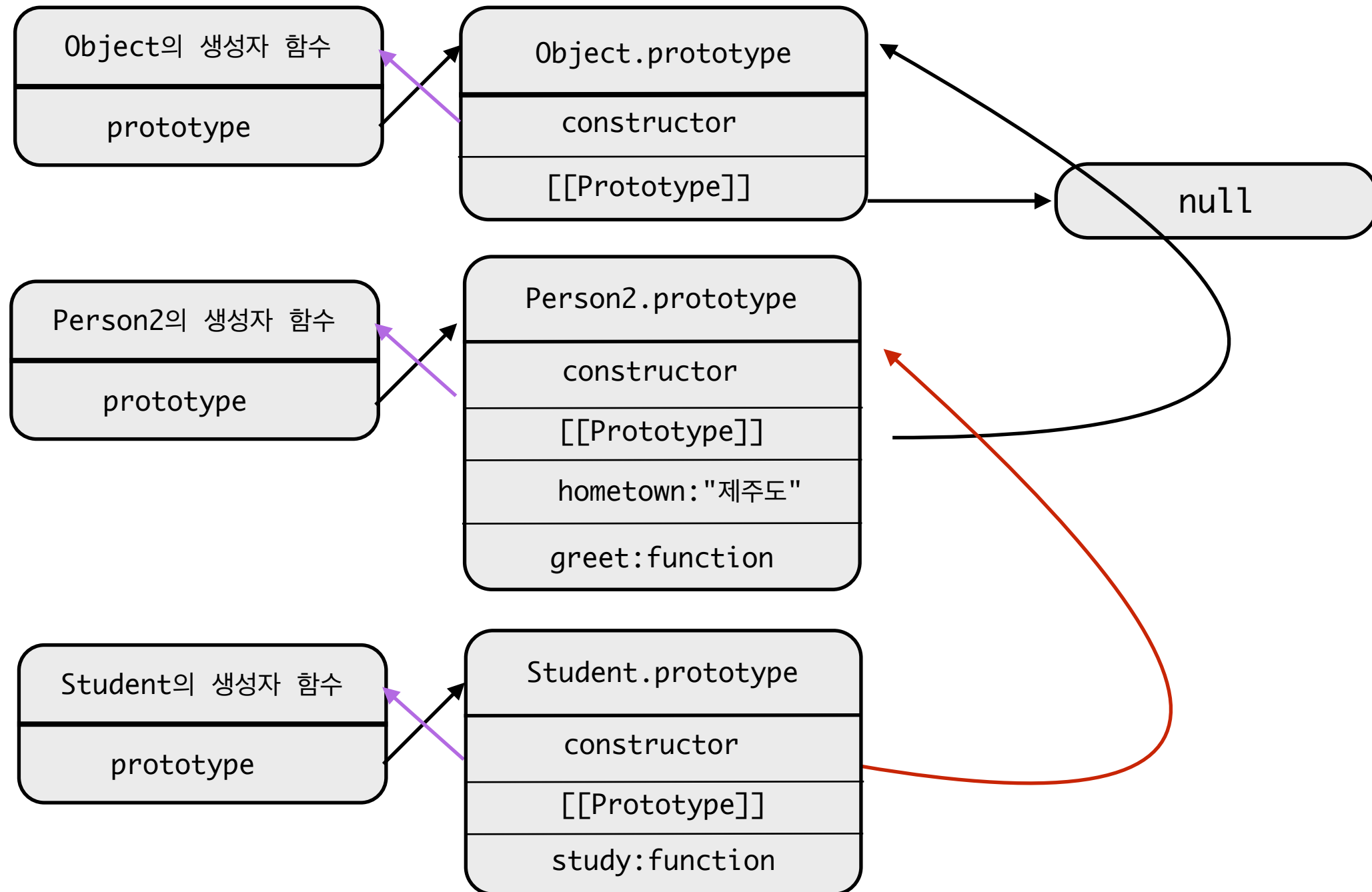
prototype에 속성 추가



prototype를 이용한 상속

```
function Student(name, age, major) {  
    Person2.call(this, name, age); // 부모 생성자 호출  
    this.major = major;  
}  
  
Student.prototype = Object.create(Person2.prototype);  
Student.prototype.constructor = Student;  
Student.prototype.study = function () {  
    console.log(`${this.name}은 ${this.major}를 공부한다`);  
};  
  
const s1 = new Student('양관식', 20, '컴퓨터공학');  
s1.greet(); // 부모 메서드  
s1.study(); // 자식 메서드
```

prototype을 이용한 상속



감사합니다.